

CNN

Prof.K.Sridhar Patnaik

Associate Professor, Dept of CSE, BIT Mesra



$$s_t = \sum_{a=0}^{\infty} x_{t-a} w_{-a} = (x * w)_t$$

- Suppose we are tracking the position of an aeroplane using a laser sensor at discrete time intervals
- Now suppose our sensor is noisy
- To obtain a less noisy estimate we would like to average several measurements
- More recent measurements are more important so we would like to take a weighted average

$$s_t = \sum_{a=0}^6 x_{t-a} w_{-a}$$

- In practice, we would only sum over a small window
- The weight array (**w**) is known as the filter
- We just slide the filter over the input and compute the value of s_t based on a window around x_t

	w_{-6}	w_{-5}	w_{-4}	w_{-3}	w_{-2}	w_{-1}	w_0
W	0.01	0.01	0.02	0.02	0.04	0.4	0.5

X	1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
---	------	------	------	------	------	------	------	------	------	------	------	------

S						1.80						
---	--	--	--	--	--	------	--	--	--	--	--	--

$$s_6 = x_6 w_0 + x_5 w_{-1} + x_4 w_{-2} + x_3 w_{-3} + x_4 w_{-4} + x_5 w_{-5} + x_6 w_{-6}$$

$$s_t = \sum_{a=0}^6 x_{t-a} w_{-a}$$

- In practice, we would only sum over a small window
- The weight array ($\mathbf{w}$) is known as the filter
- We just slide the filter over the input and compute the value of s_t based on a window around x_t

	w_{-6}	w_{-5}	w_{-4}	w_{-3}	w_{-2}	w_{-1}	w_0
W	0.01	0.01	0.02	0.02	0.04	0.4	0.5

X	1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
---	------	------	------	------	------	------	------	------	------	------	------	------

S						1.80	1.96				
---	--	--	--	--	--	------	------	--	--	--	--

$$s_6 = x_6 w_0 + x_5 w_{-1} + x_4 w_{-2} + x_3 w_{-3} + x_4 w_{-4} + x_5 w_{-5} + x_6 w_{-6}$$

$$s_t = \sum_{a=0}^6 x_{t-a} w_{-a}$$

- In practice, we would only sum over a small window
- The weight array (**w**) is known as the filter
- We just slide the filter over the input and compute the value of s_t based on a window around x_t

	w_{-6} w_{-5} w_{-4} w_{-3} w_{-2} w_{-1} w_0																							
W	<table><tr><td>0.01</td><td>0.01</td><td>0.02</td><td>0.02</td><td>0.04</td><td>0.4</td><td>0.5</td></tr></table>							0.01	0.01	0.02	0.02	0.04	0.4	0.5										
0.01	0.01	0.02	0.02	0.04	0.4	0.5																		
X	<table><tr><td>1.00</td><td>1.10</td><td>1.20</td><td>1.40</td><td>1.70</td><td>1.80</td><td>1.90</td><td>2.10</td><td>2.20</td><td>2.40</td><td>2.50</td><td>2.70</td></tr></table>												1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70													
S	<table><tr><td>1.80</td><td>1.96</td><td>2.11</td><td>2.16</td><td>2.28</td><td>2.42</td></tr></table>						1.80	1.96	2.11	2.16	2.28	2.42												
1.80	1.96	2.11	2.16	2.28	2.42																			

$$s_6 = x_6 w_0 + x_5 w_{-1} + x_4 w_{-2} + x_3 w_{-3} + x_4 w_{-4} + x_5 w_{-5} + x_6 w_{-6}$$

$$s_t = \sum_{a=0}^6 x_{t-a} w_{-a}$$

	w_{-6}	w_{-5}	w_{-4}	w_{-3}	w_{-2}	w_{-1}	w_0					
W	0.01	0.01	0.02	0.02	0.04	0.4	0.5					
X	1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
S							1.80	1.96	2.11	2.16	2.28	2.42

$$s_6 = x_6 w_0 + x_5 w_{-1} + x_4 w_{-2} + x_3 w_{-3} + x_4 w_{-4} + x_5 w_{-5} + x_6 w_{-6}$$

- In practice, we would only sum over a small window
- The weight array ($\mathbf{w}$) is known as the filter
- We just slide the filter over the input and compute the value of s_t based on a window around x_t
- Here the input (and the kernel) is one dimensional
- Can we use a convolutional operation on a 2D input also?



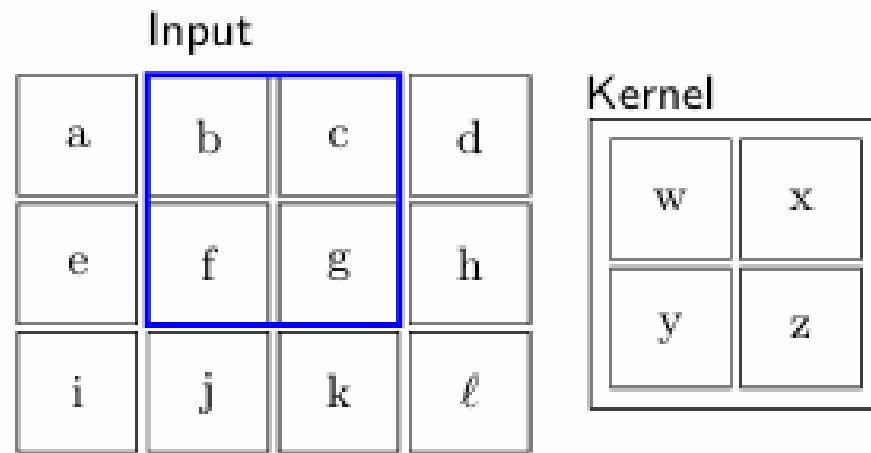
$$S_{ij} = (I * K)_{ij} = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I_{i+a, j+b} K_{a,b}$$

- We can think of images as 2D inputs
- We would now like to use a 2D filter ($m \times n$)
- First let us see what the 2D formula looks like
- This formula looks at all the preceding neighbours ($i - a, j - b$)
- In practice, we use the following formula which looks at the succeeding neighbours

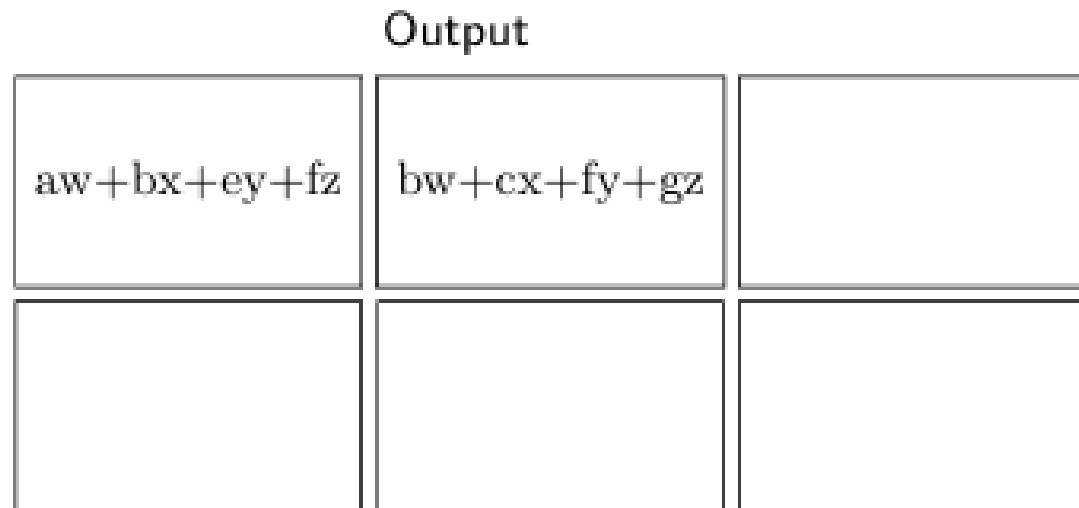
Input				Kernel	
a	b	c	d	w	x
e	f	g	h	y	z
i	j	k	ℓ		

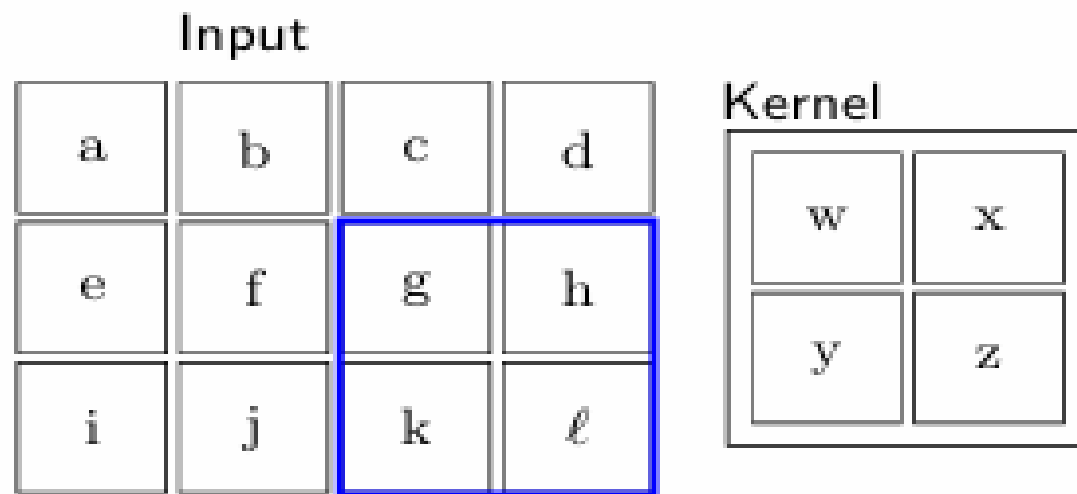
- Let us apply this idea to a toy example and see the results

Output		
$aw+bx+ey+fz$		



- Let us apply this idea to a toy example and see the results



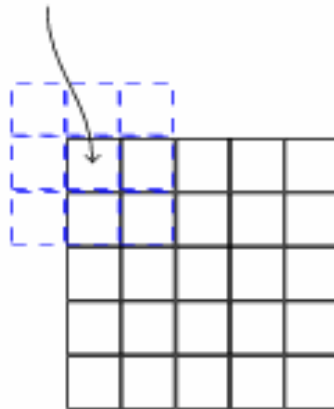


Output

$aw + bx + ey + fz$	$bw + cx + fy + gz$	$cw + dx + gy + hz$
$ew + fx + iy + jz$	$fw + gx + jy + kz$	$gw + hx + ky + \ell z$

$$S_{ij} = (I * K)_{ij} = \sum_{a=\lfloor -\frac{m}{2} \rfloor}^{\lfloor \frac{m}{2} \rfloor} \sum_{b=\lfloor -\frac{n}{2} \rfloor}^{\lfloor \frac{n}{2} \rfloor} I_{i-a, j-b} K_{\frac{m}{2}+a, \frac{n}{2}+b}$$

pixel of interest



- For the rest of the discussion we will use the following formula for convolution
- In other words we will assume that the kernel is centered on the pixel of interest
- So we will be looking at both preceding and succeeding neighbors

Let us see some examples of 2D convolutions applied to images



$$\begin{matrix} & 1 & 1 & 1 \\ * & 1 & 1 & 1 \\ & 1 & 1 & 1 \end{matrix} =$$



blurs the image



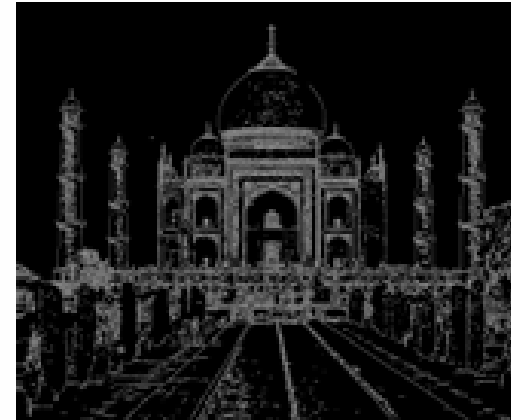
$$\begin{matrix} & 0 & -1 & 0 \\ * & -1 & 5 & -1 \\ & 0 & -1 & 0 \end{matrix} =$$



sharpens the image



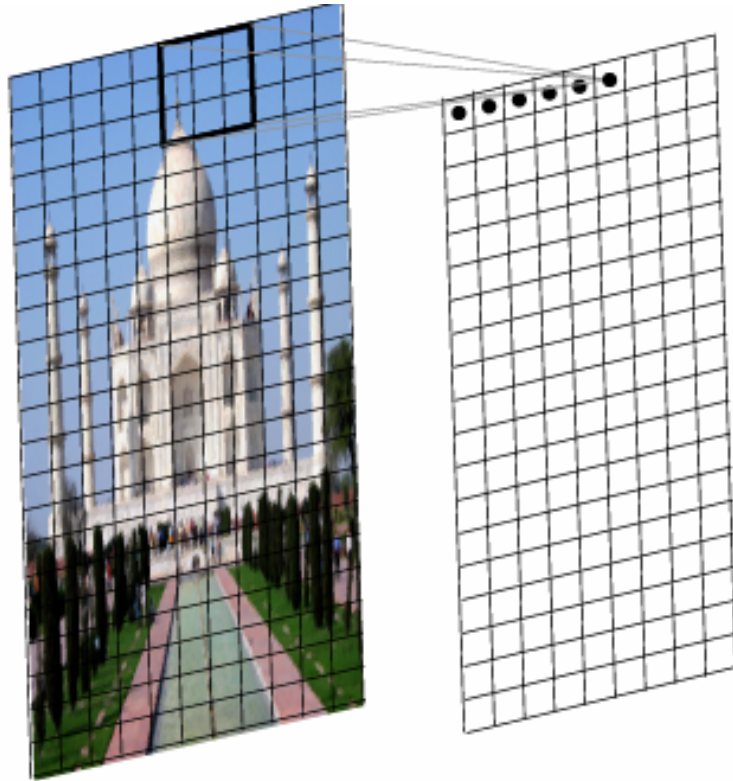
$$* \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix} =$$



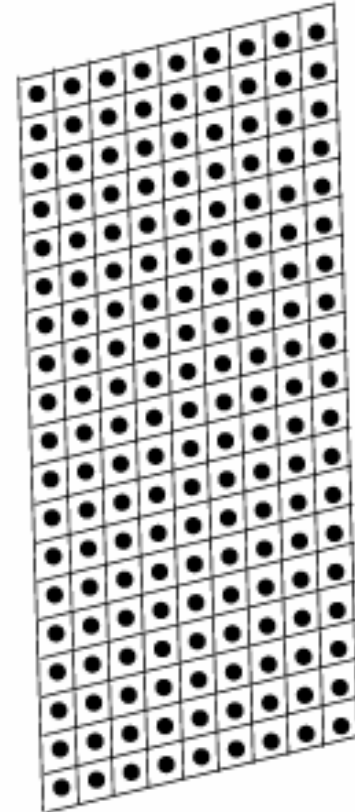
detects the edges

Edges means change in intensity

We will now see a working example of 2D convolution



- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output



- We just slide the kernel over the input image
- Each time we slide the kernel we get one value in the output
- The resulting output is called a feature map.
- We can use multiple filters to get multiple feature maps.

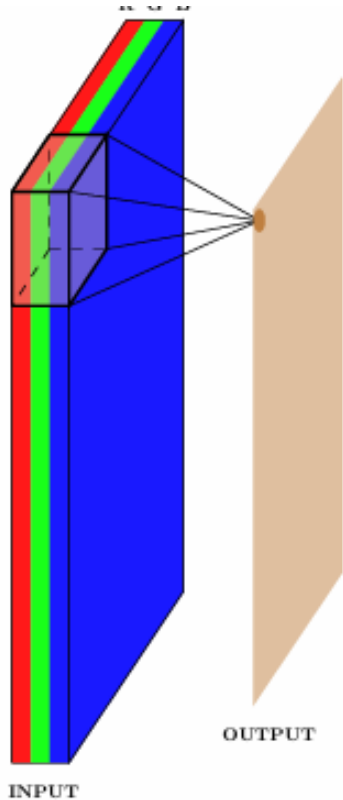
Question

- In the 1D case, we slide a one dimensional filter over a one dimensional input
- In the 2D case, we slide a two dimensional filter over a two dimensional input

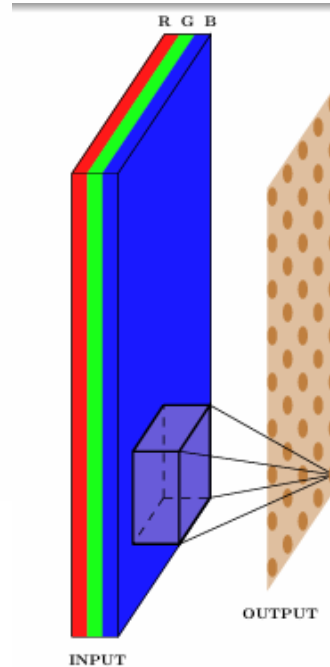
A	B	C	B	A	B	C
---	---	---	---	---	---	---

a	b	c	d
e	f	g	h
i	j	k	l

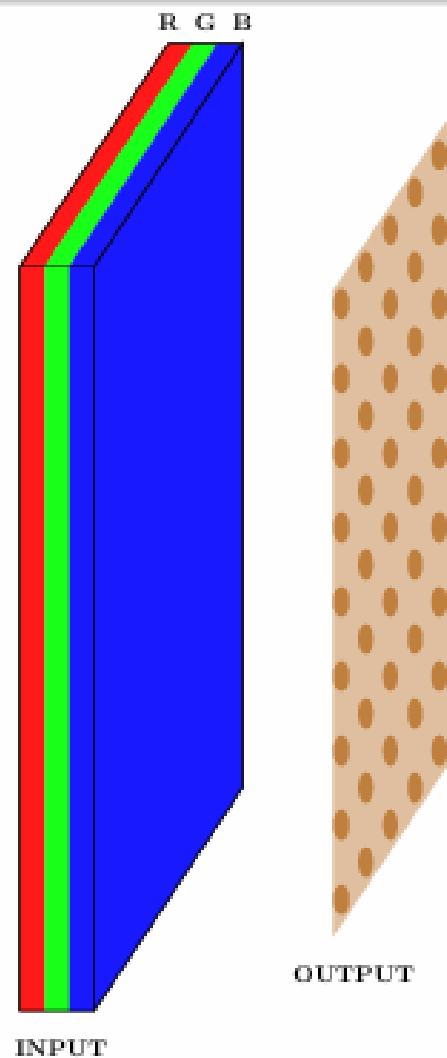
What would happen in the 3D case?



- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation



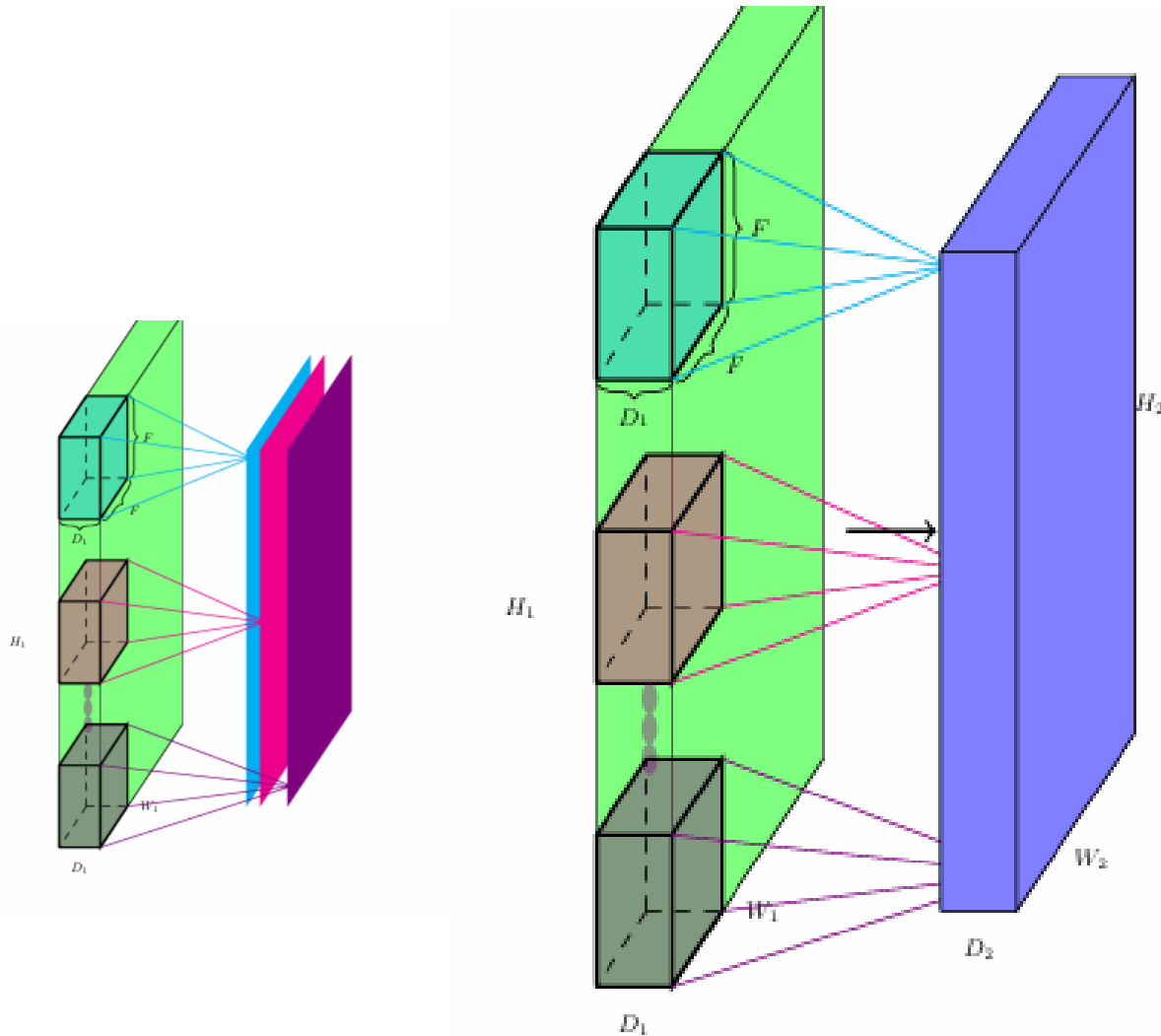
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation
- Note that in this lecture we will assume that the filter always extends to the depth of the image



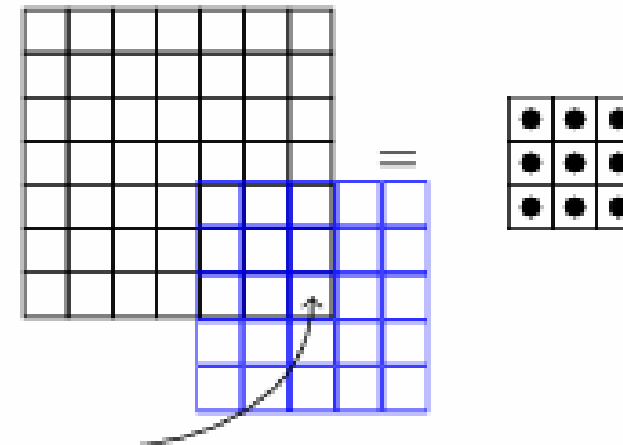
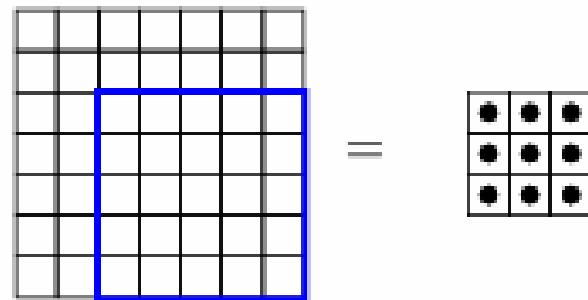
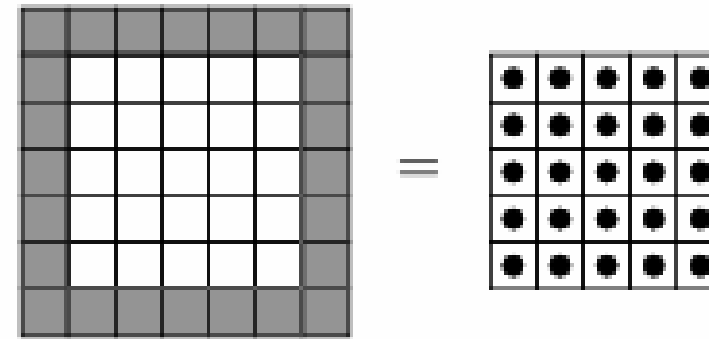
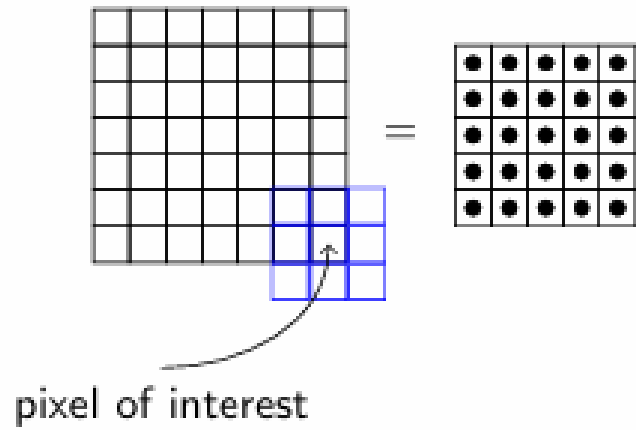
- What would a 3D filter look like?
- It will be 3D and we will refer to it as a volume
- Once again we will slide the volume over the 3D input and compute the convolution operation
- Note that in this lecture we will assume that the filter always extends to the depth of the image
- In effect, we are doing a 2D convolution operation on a 3D input (because the filter moves along the height and the width but not along the depth)
- As a result the output will be 2D (only width and height, no depth)
- Once again we can apply multiple filters to get multiple feature maps

Relation between input size, output size and filter size

- So far we have not said anything explicit about the dimensions of the
 - ① inputs
 - ② filters
 - ③ outputsand the relations between them
- We will see how they are related but before that we will define a few quantities

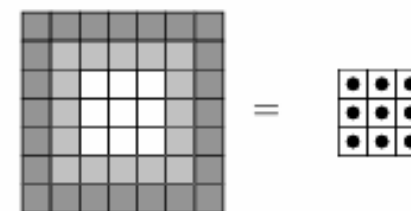


- We first define the following quantities
- Width (W_1), Height (H_1) and Depth (D_1) of the original input
- The Stride S (We will come back to this later)
- The number of filters K
- The spatial extent (F) of each filter (the depth of each filter is same as the depth of each input)
- The output is $W_2 \times H_2 \times D_2$ (we will soon see a formula for computing W_2 , H_2 and D_2)



pixel of interest

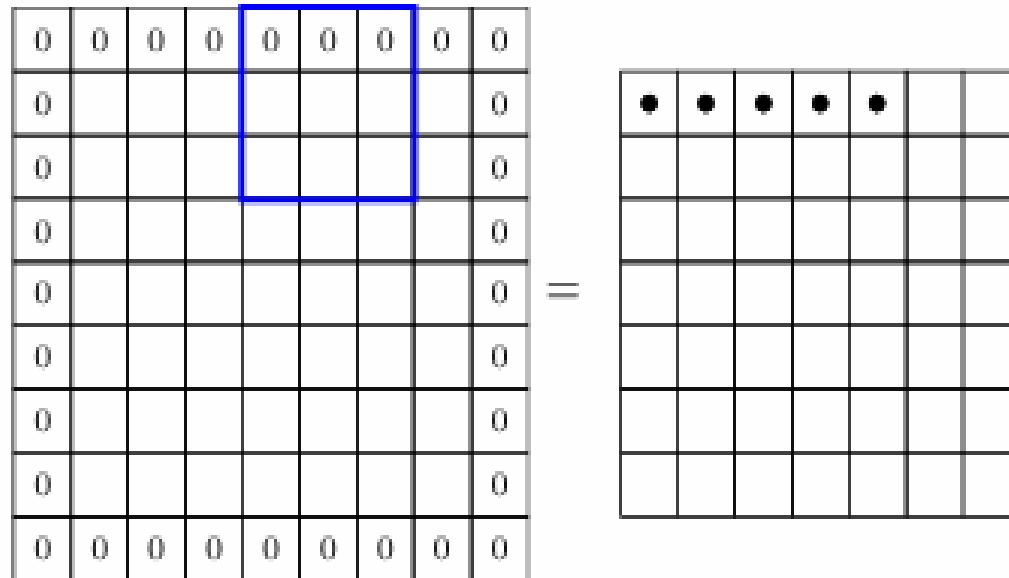
- Let us compute the dimension (W_2, H_2) of the output
- Notice that we can't place the kernel at the corners as it will cross the input boundary
- This is true for all the shaded points (the kernel crosses the input boundary)
- This results in an output which is of smaller dimensions than the input
- As the size of the kernel increases, this becomes true for even more pixels
- For example, let's consider a 5×5 kernel
- We have an even smaller output now



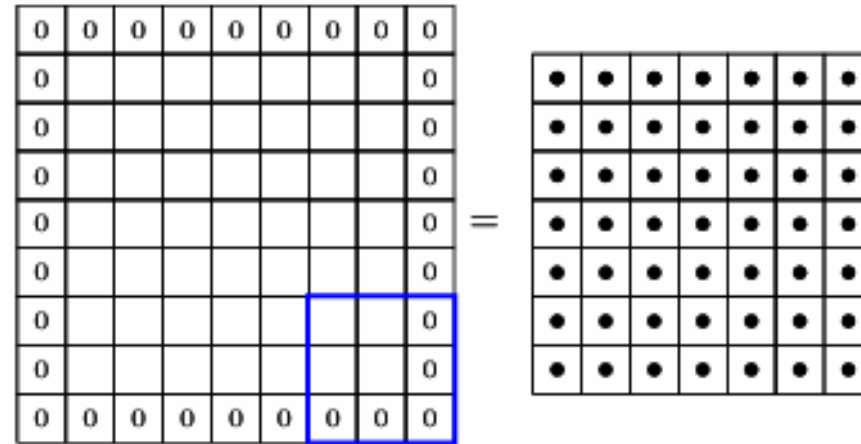
In general, $W_2 = W_1 - F + 1$

$H_2 = H_1 - F + 1$

We will refine this formula further



- What if we want the output to be of same size as the input?
- We can use something known as padding
- Pad the inputs with appropriate number of 0 inputs so that you can now apply the kernel at the corners
- Let us use pad $P = 1$ with a 3×3 kernel
- This means we will add one row and one column of 0 inputs at the top, bottom, left and right



We now have,

$$W_2 = W_1 - F + 2P + 1$$

$$H_2 = H_1 - F + 2P + 1$$

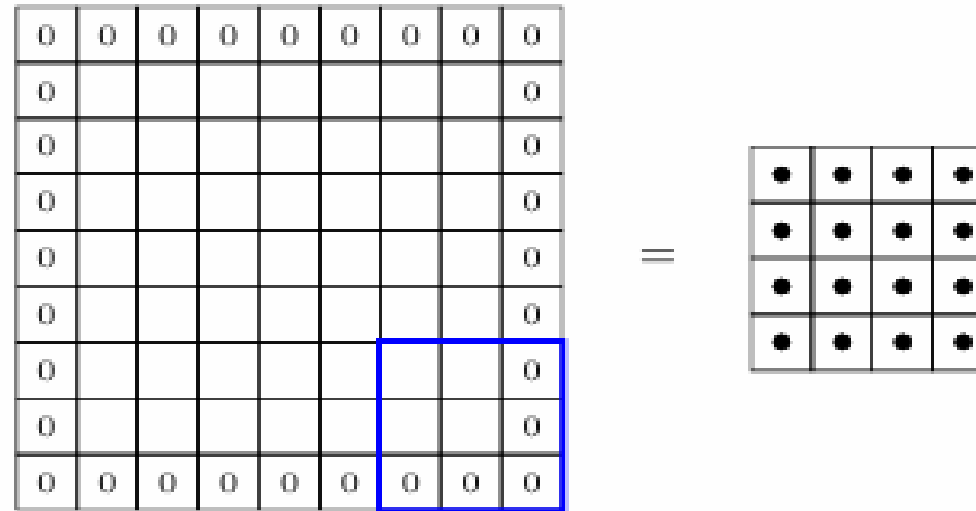
We will refine this formula further

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

=

•	•	•	•

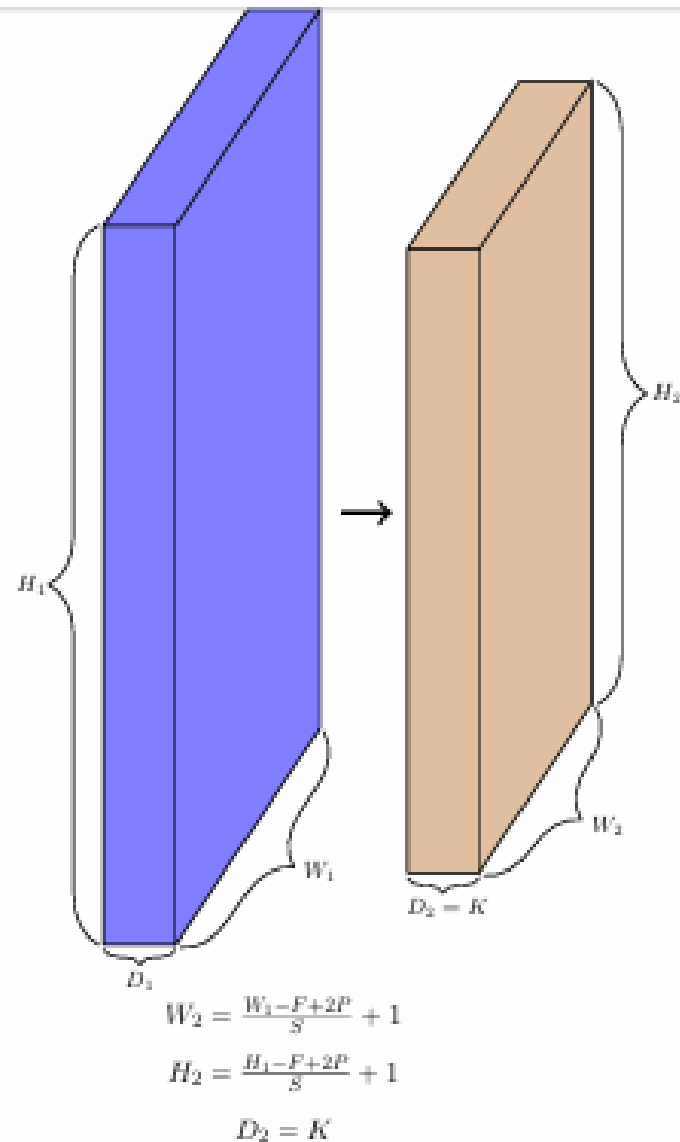
- What does the stride S do?
- It defines the intervals at which the filter is applied (here $S = 2$)
- Here, we are essentially skipping every 2nd pixel which will again result in an output which is of smaller dimensions



So what should our final formula look like,

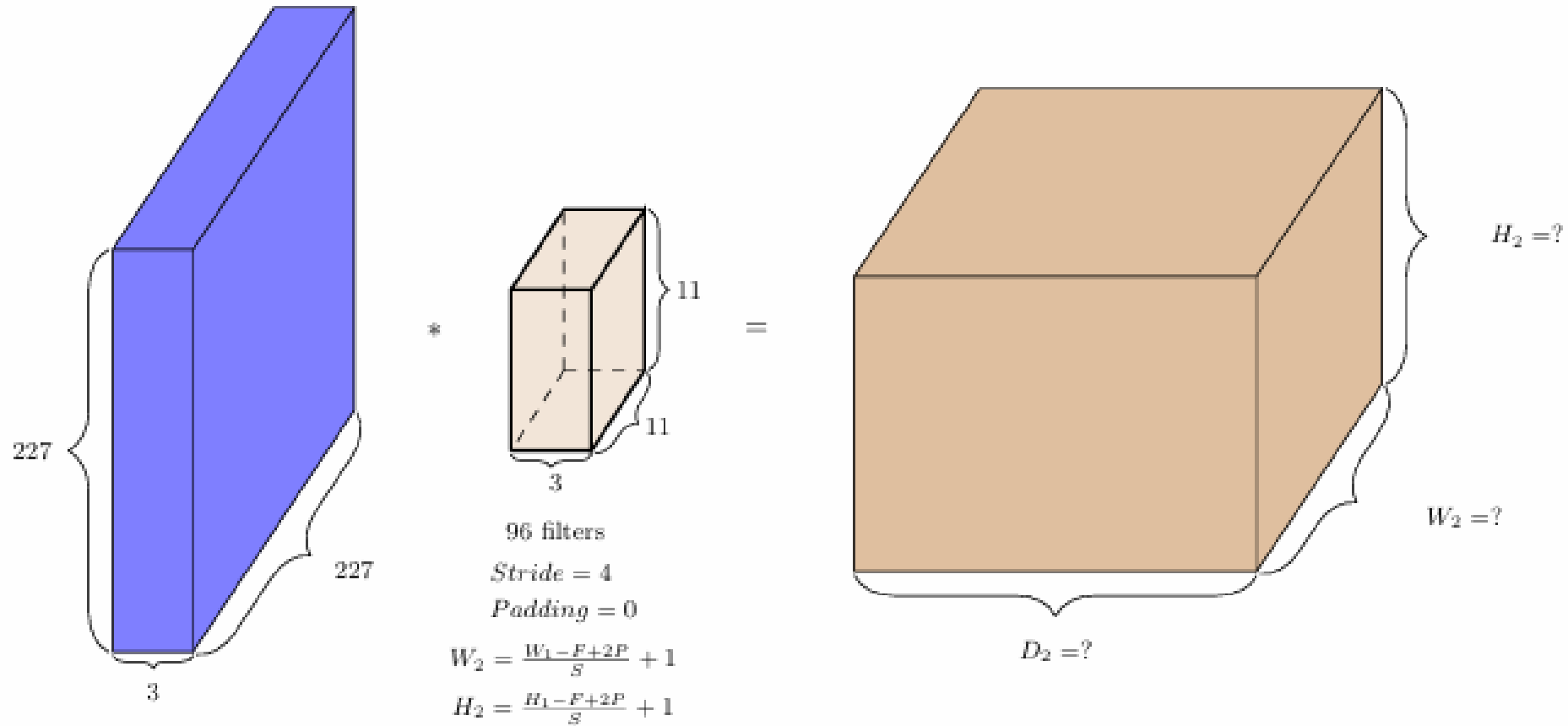
$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

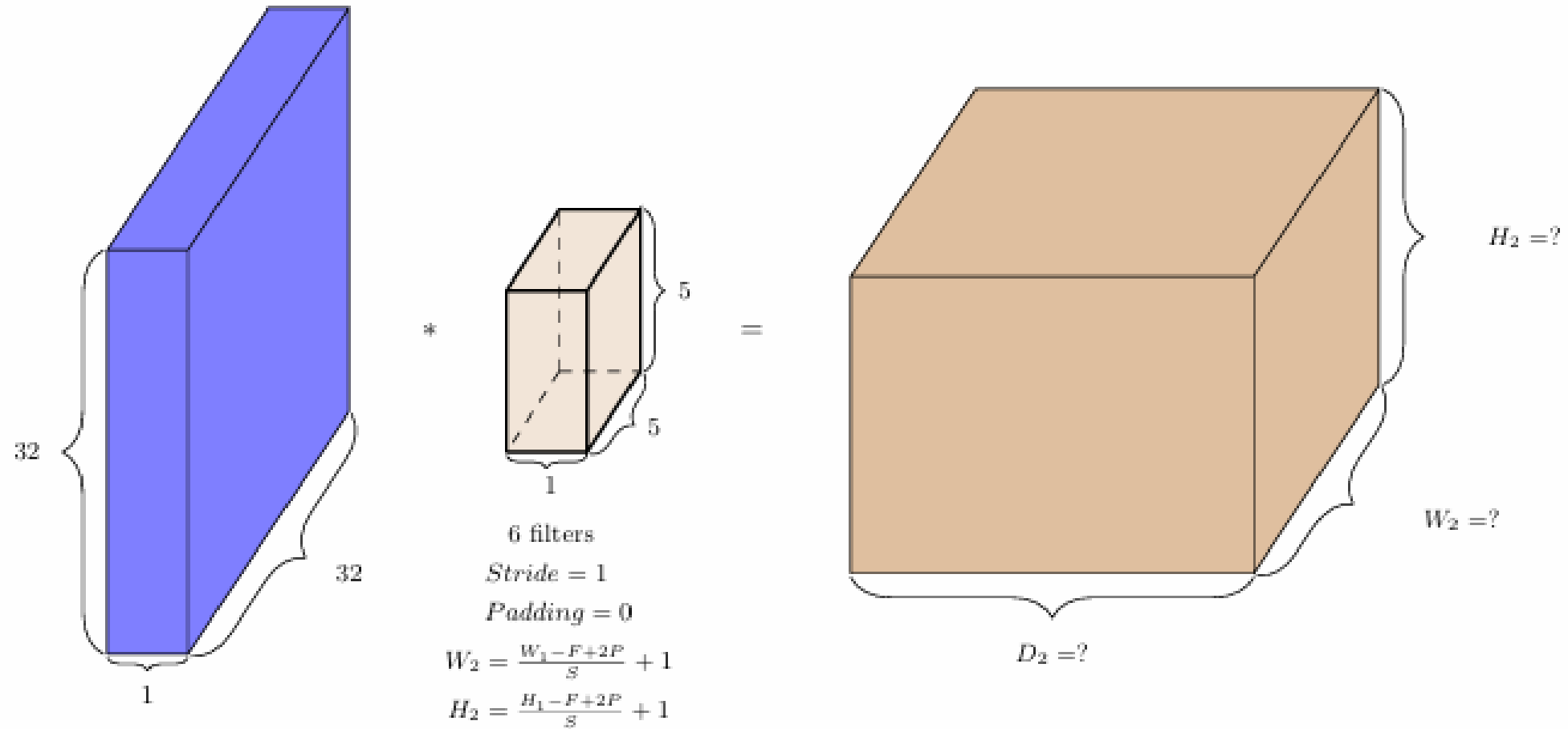
$$H_2 = \frac{H_1 - F + 2P}{S} + 1$$



- Finally, coming to the depth of the output.
- Each filter gives us one 2D output.
- K filters will give us K such 2D outputs
- We can think of the resulting output as $K \times W_2 \times H_2$ volume
- Thus $D_2 = K$

Exercise

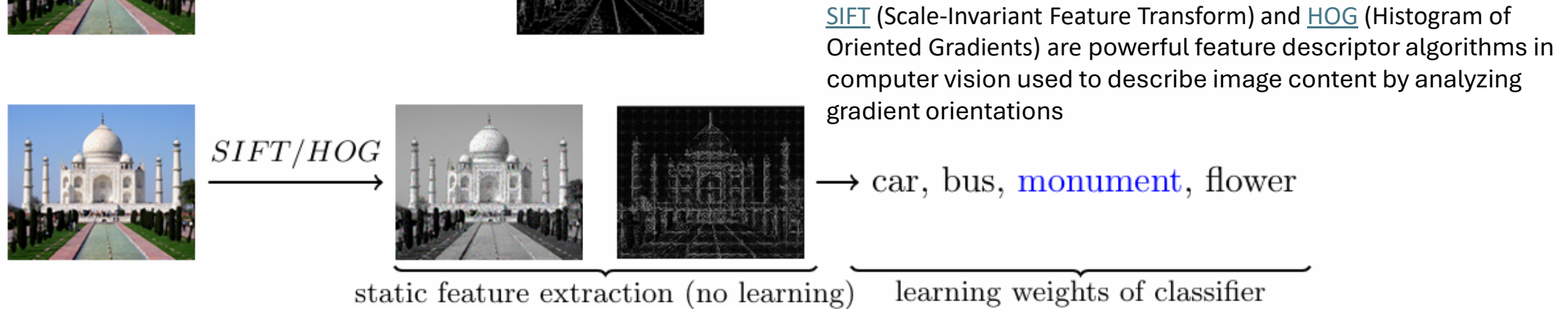
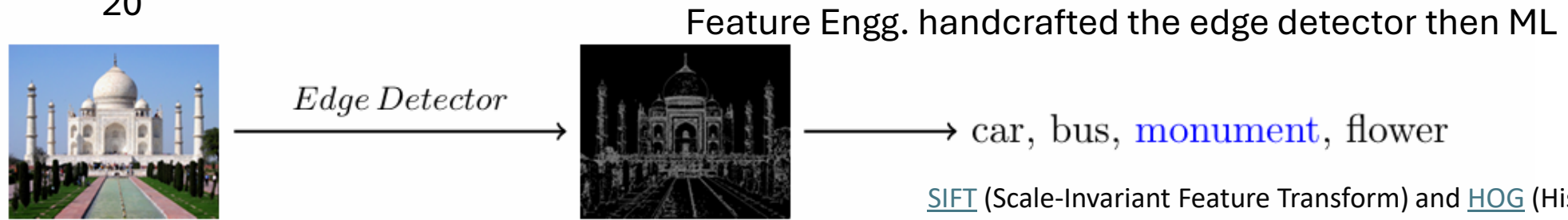
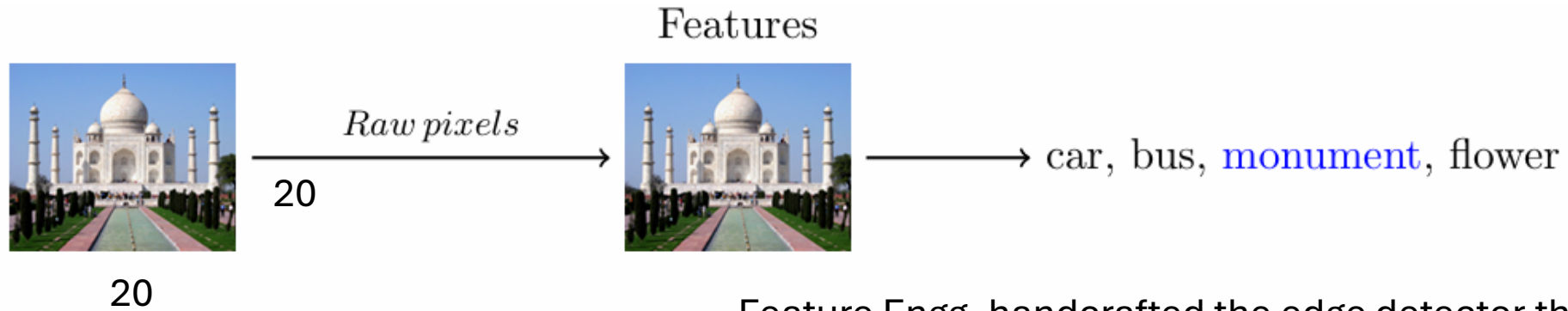




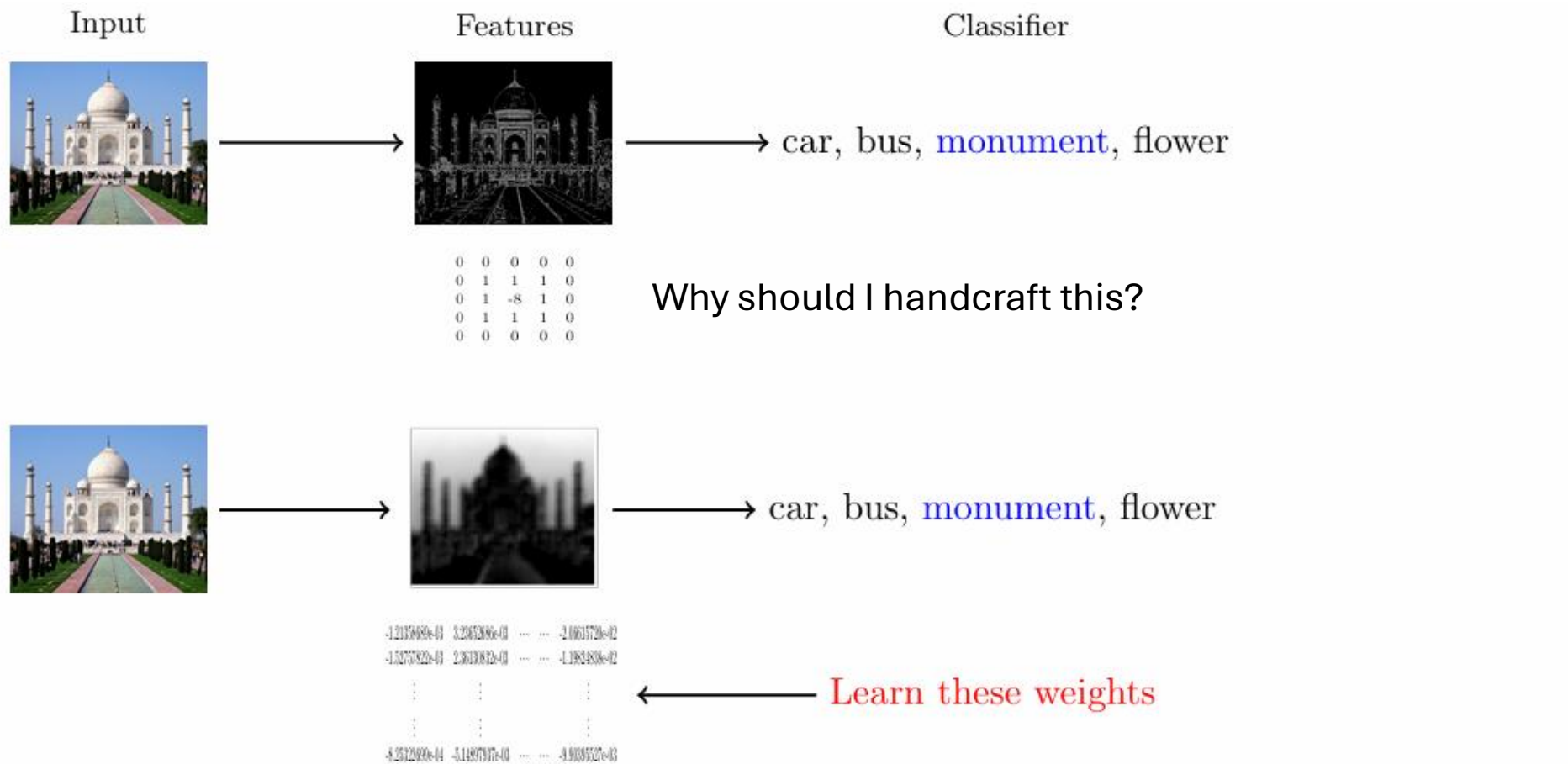
Convolutional Neural Networks

Putting things into perspective

- What is the connection between this operation (convolution) and neural networks?
- We will try to understand this by considering the task of “image classification”



Feature extraction is deterministic

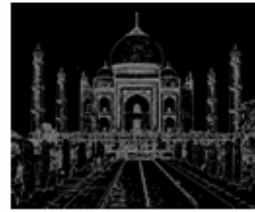


- Instead of using handcrafted kernels such as edge detectors **can we learn meaningful kernels/filters in addition to learning the weights of the classifier?**

Input



Features



0	0	0	0	0
0	1	1	1	0
0	1	-8	1	0
0	1	1	1	0
0	0	0	0	0

Classifier

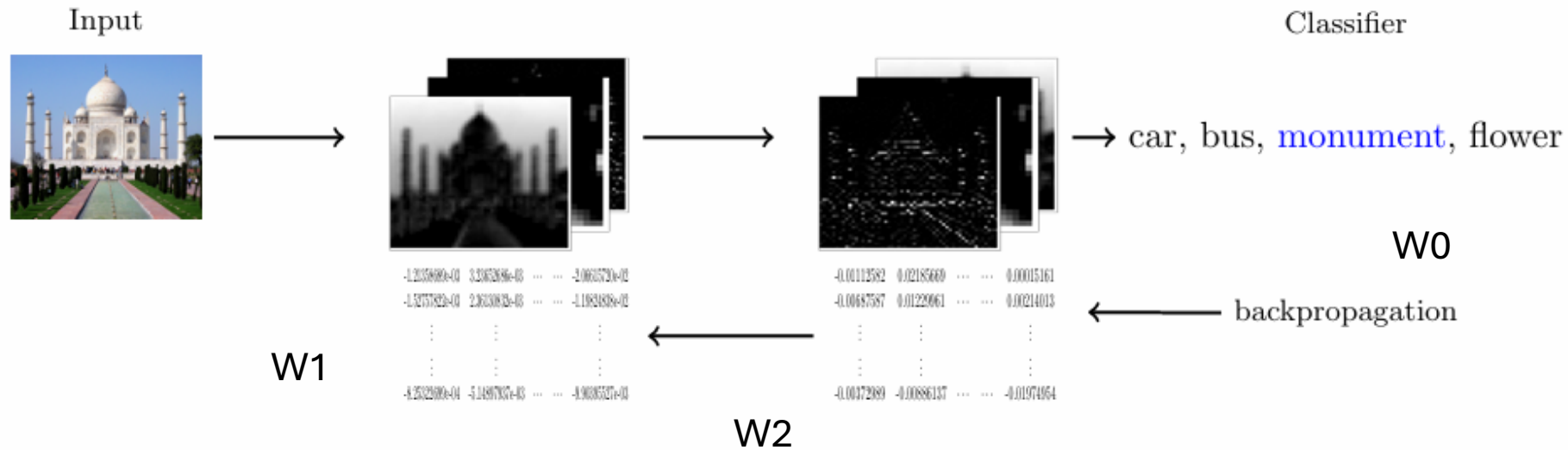
car, bus, **monument**, flower



-0.01871333	-0.01075948	...	...	0.04684572
0.00104325	0.01935837	...	...	0.01016542
:	:			:
:	:			:
0.03008777	0.00335217	...	...	-0.02791128

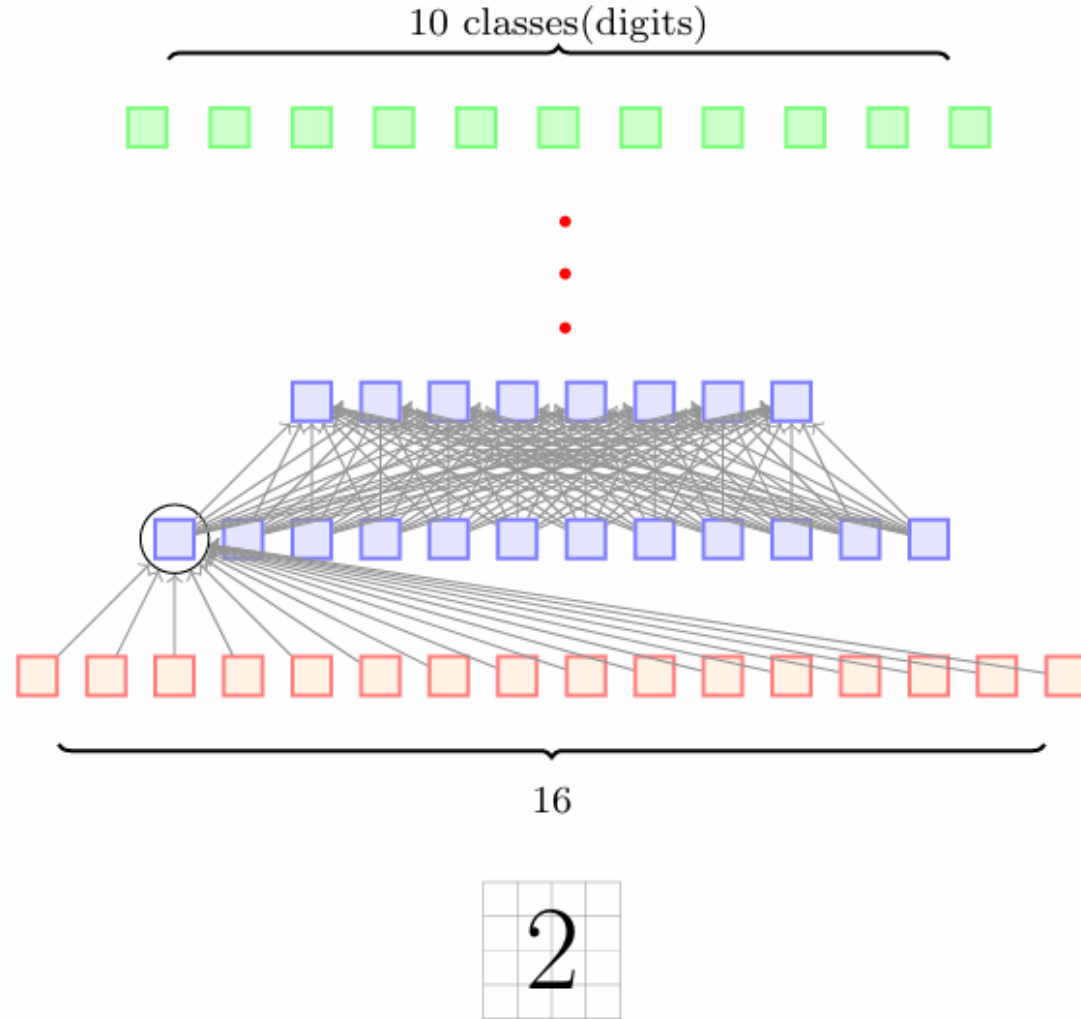
car, bus, **monument**, flower

- **Even better:** Instead of using handcrafted kernels (such as edge detectors) can we learn **multiple** meaningful kernels/filters in addition to learning the weights of the classifier?

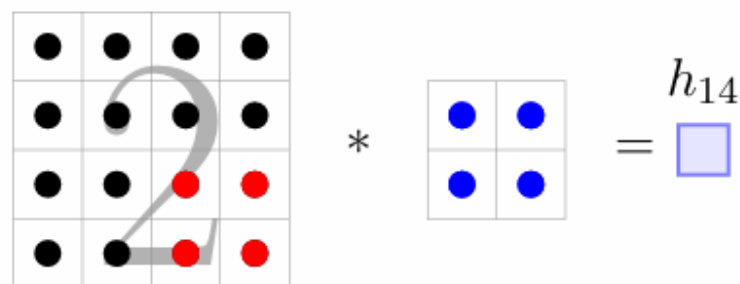
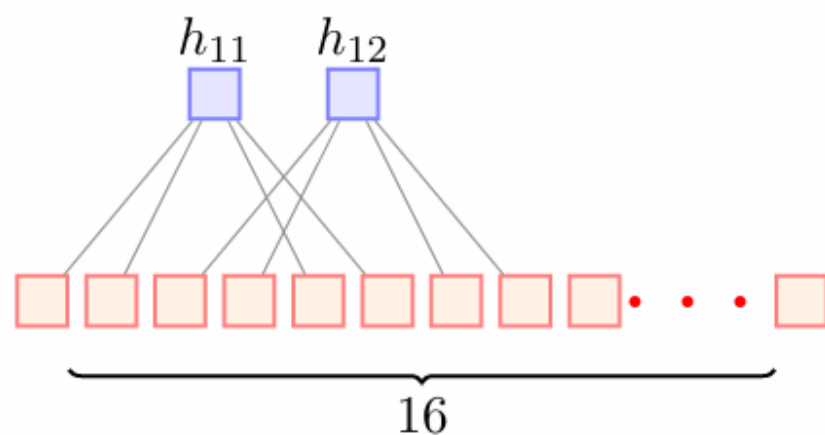


- Can we learn multiple **layers** of meaningful kernels/filters in addition to learning the weights of the classifier?
- Yes, we can !
- Simply by treating these kernels as parameters and learning them in addition to the weights of the classifier (using back propagation)
- Such a network is called a Convolutional Neural Network.

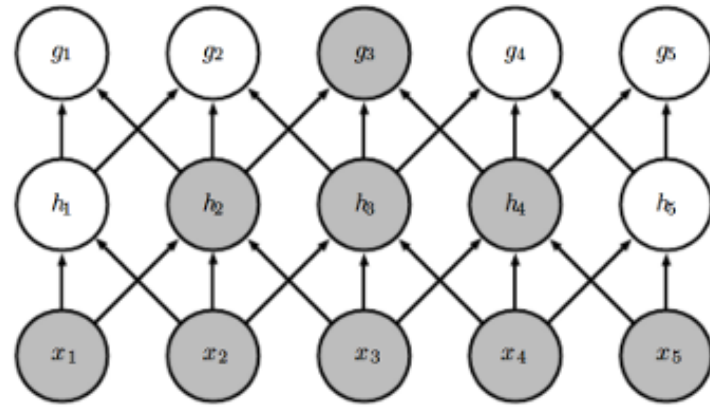
- Okay, I get it that the idea is to learn the kernel/filters by just treating them as parameters of the classification model
- But how is this different from a regular feedforward neural network
- Let us see



- This is what a regular feed-forward neural network will look like
- There are many dense connections here
- For example all the 16 input neurons are contributing to the computation of h_{11}
- Contrast this to what happens in the case of convolution

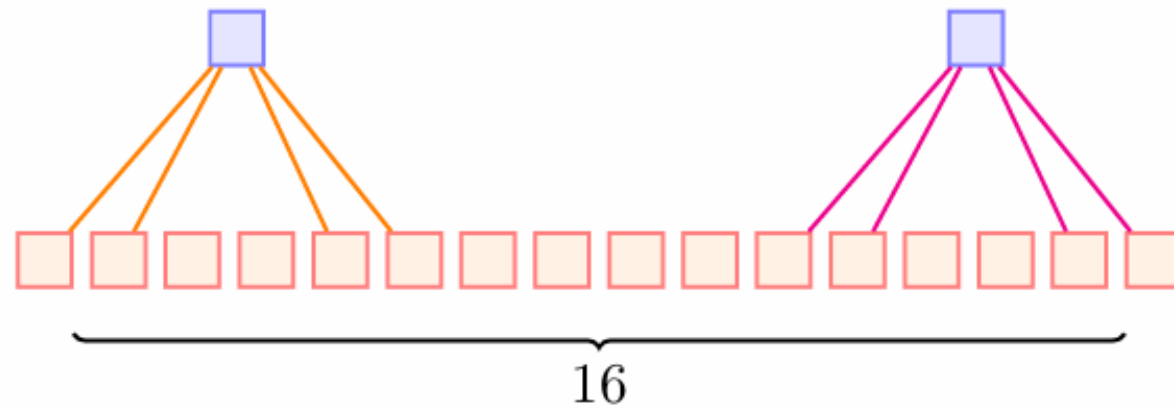


- Only a few local neurons participate in the computation of h_{11}
- For example, only pixels 1, 2, 5, 6 contribute to h_{11}
- The connections are much sparser
- We are taking advantage of the structure of the image (interactions between neighboring pixels are more interesting)
- This **sparse connectivity** reduces the number of parameters in the model



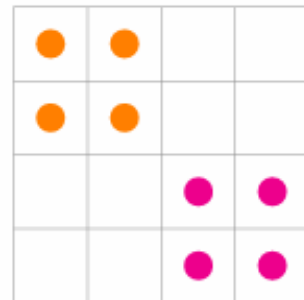
- But is sparse connectivity really good thing ?
- Aren't we losing information (by losing interactions between some input pixels)
- Well, not really
- The two highlighted neurons (x_1 & x_5)* do not interact in *layer 1*
- But they indirectly contribute to the computation of g_3 and hence interact indirectly

* Goodfellow-et-al-2016



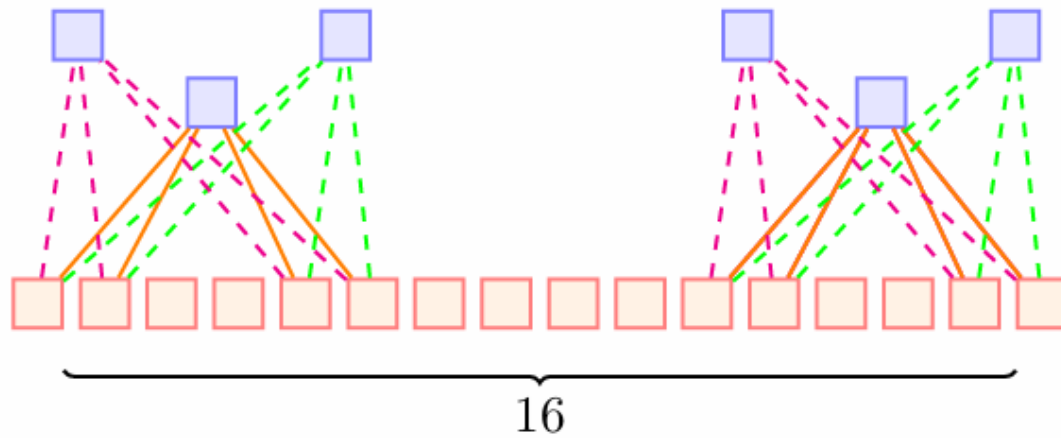
● Kernel 1

● Kernel 2



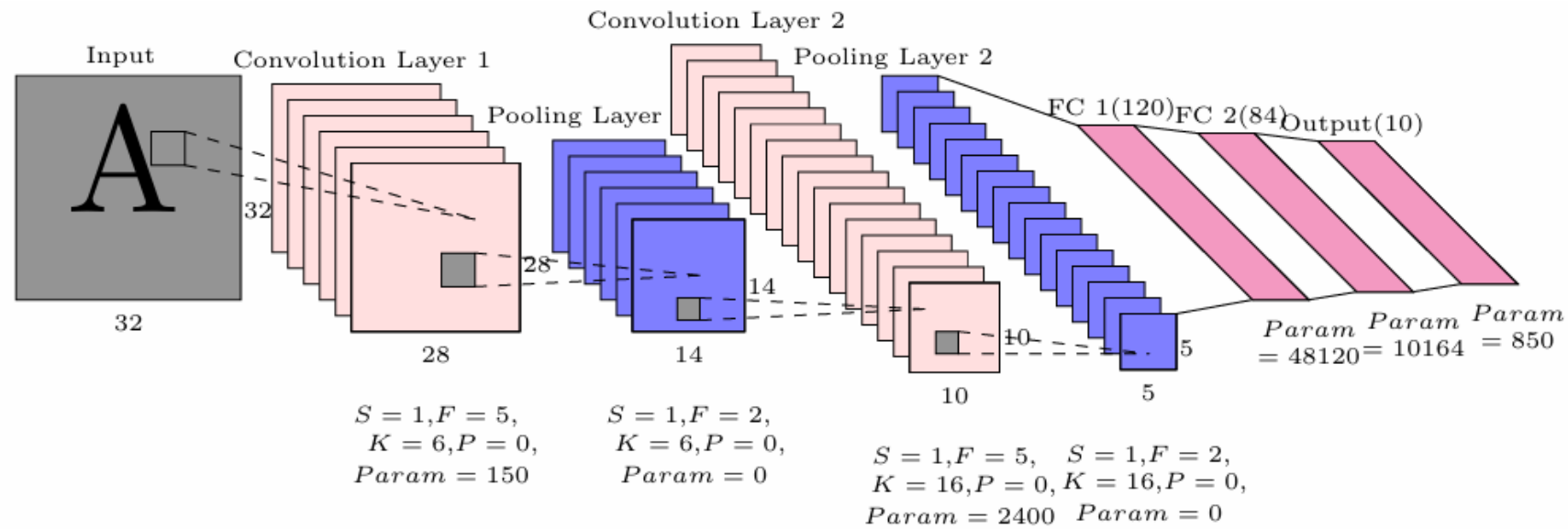
4x4 Image

- Another characteristic of CNNs is **weight sharing**
- Consider the following network
- Do we want the kernel weights to be different for different portions of the image?
- Imagine that we are trying to learn a kernel that detects edges
- Shouldn't we be applying the same kernel at all the portions of the image?



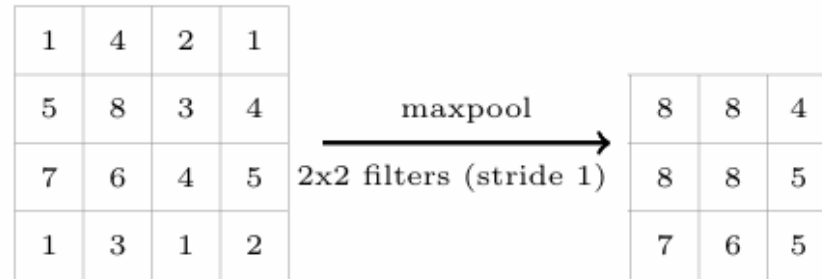
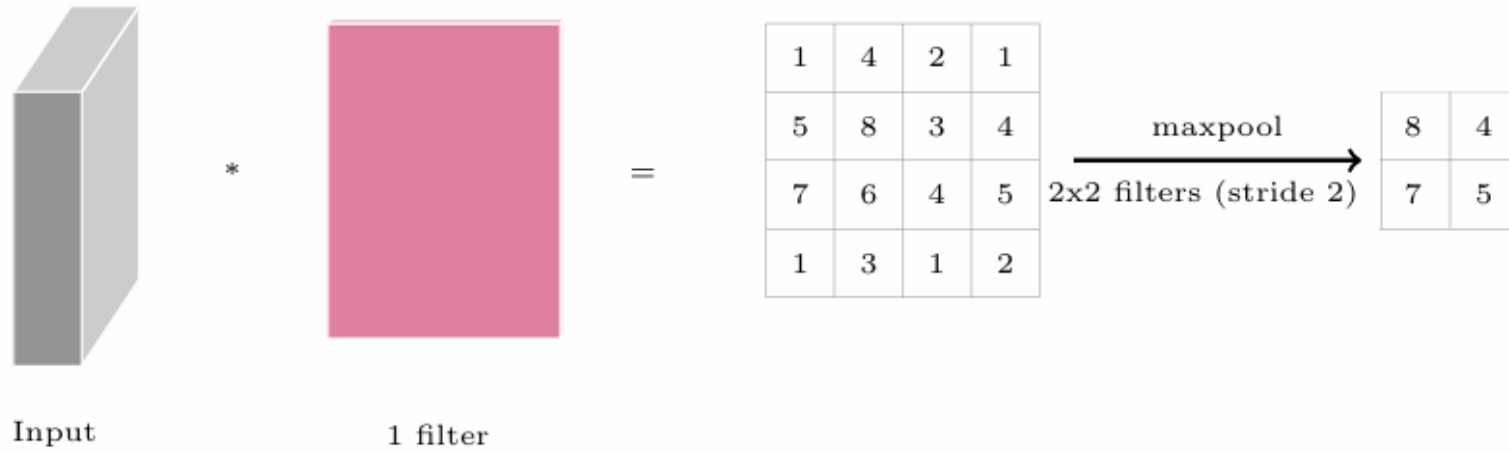
- In other words shouldn't the *orange* and *pink* kernels be the same
- Yes, indeed
- This would make the job of learning easier (instead of trying to learn the same weights/kernels at different locations again and again)
- But does that mean we can have only one kernel?
- No, we can have many such kernels but the kernels will be shared by all locations in the image
- This is called “weight sharing”

- So far, we have focused only on the convolution operation
- Let us see what a full convolutional neural network looks like



- It has alternate convolution and pooling layers
- What does a pooling layer do?
- Let us see

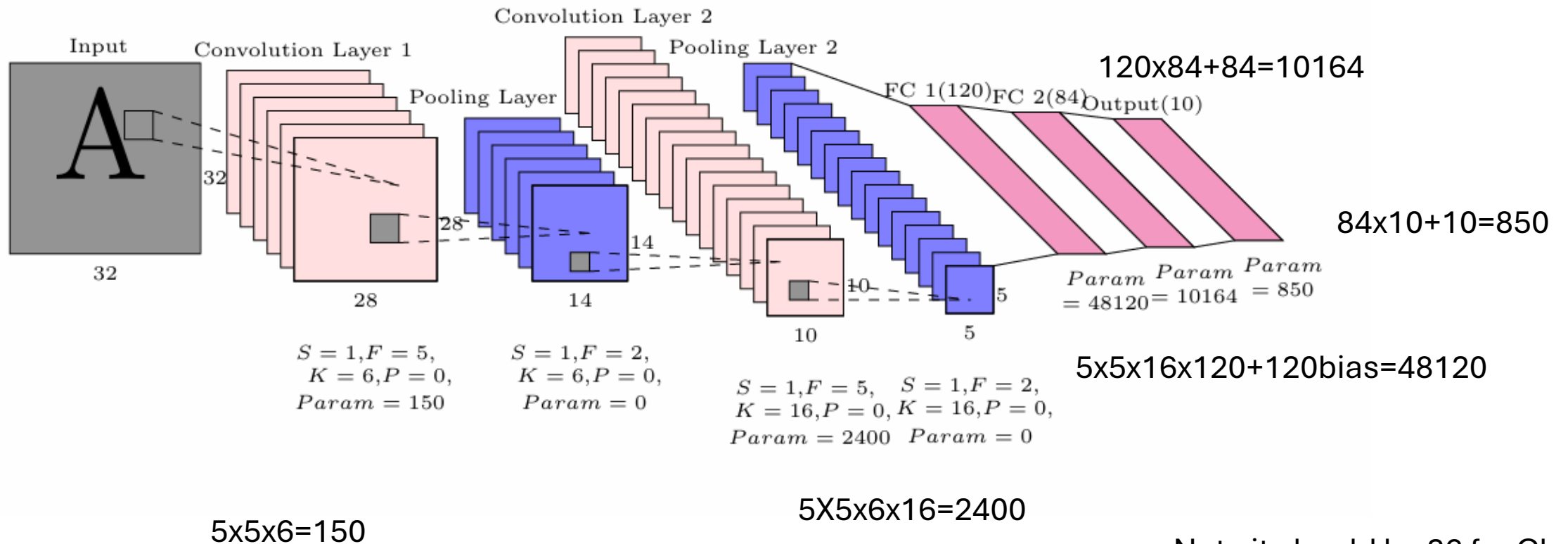
Pooling



- Instead of max pooling we can also do average pooling

We will now see some case studies where convolution neural networks have been successful

LeNet-5 for handwritten character recognition

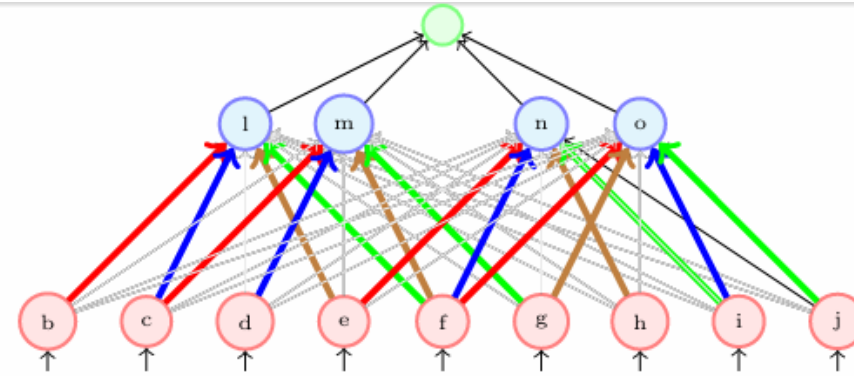


Parameters in a Convolutional Neural Network (CNN) are calculated per layer by multiplying the filter dimensions (width $\times$ height) by the number of input channels, adding one bias term per filter, and multiplying by the total number of filters:

$$\text{Parameters} = ((\text{filter_w} \times \text{filter_h} \times \text{input_channels}) + 1) \times \text{filters}$$

Input		
b	c	d
e	f	g
h	i	j

Kernel	
w	x
y	z

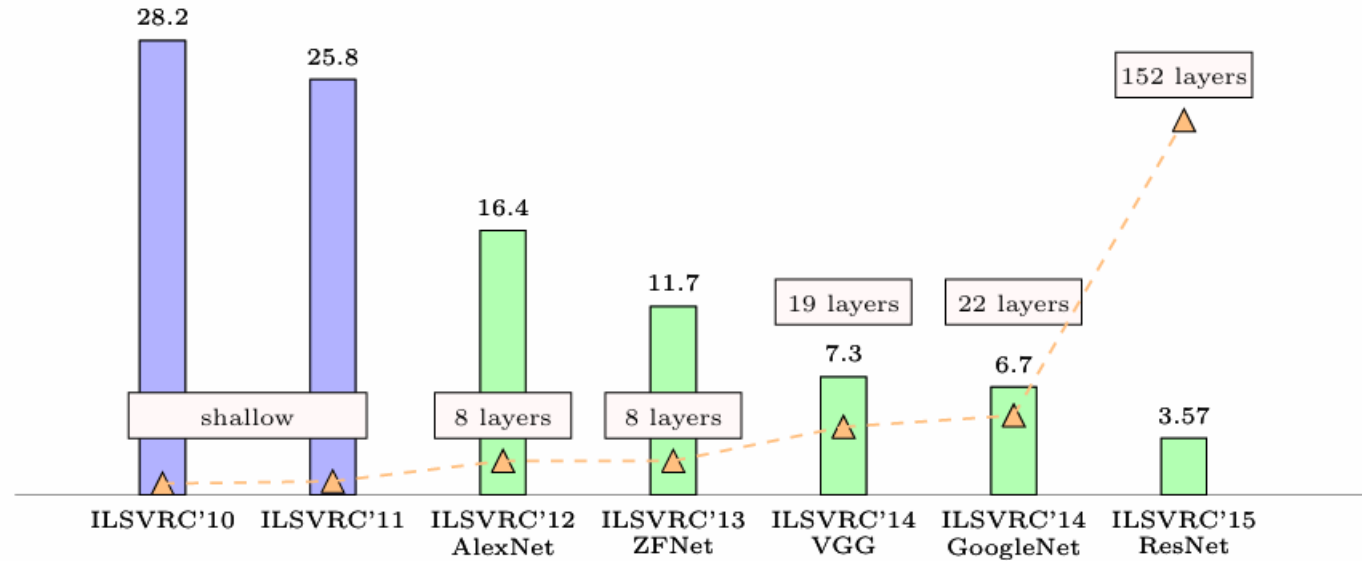


- We can thus train a convolution neural network using backpropagation by thinking of it as a feedforward neural network with sparse connections
- A CNN can be implemented as a feedforward neural network
- wherein only a few weights(in color) are active
- the rest of the weights (in gray) are zero

CNNs (success stories on ImageNet)

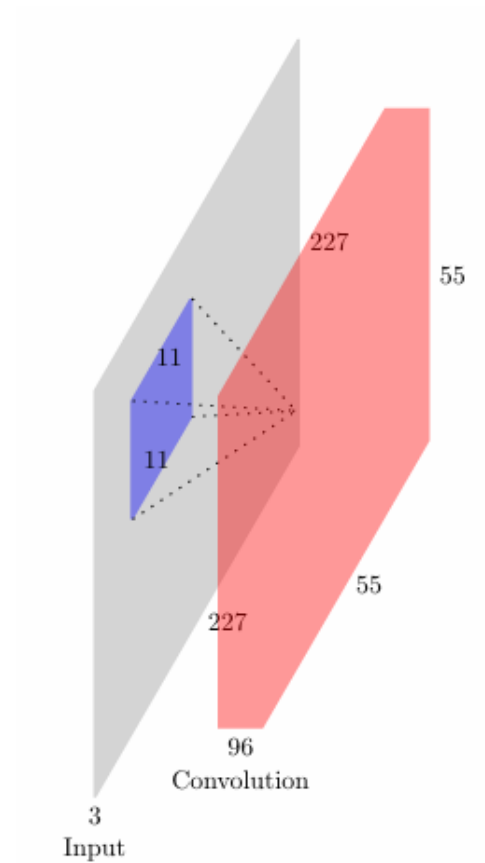
ImageNet Success Stories(roadmap for rest of the talk)

- AlexNet
- ZFNet
- VGGNet



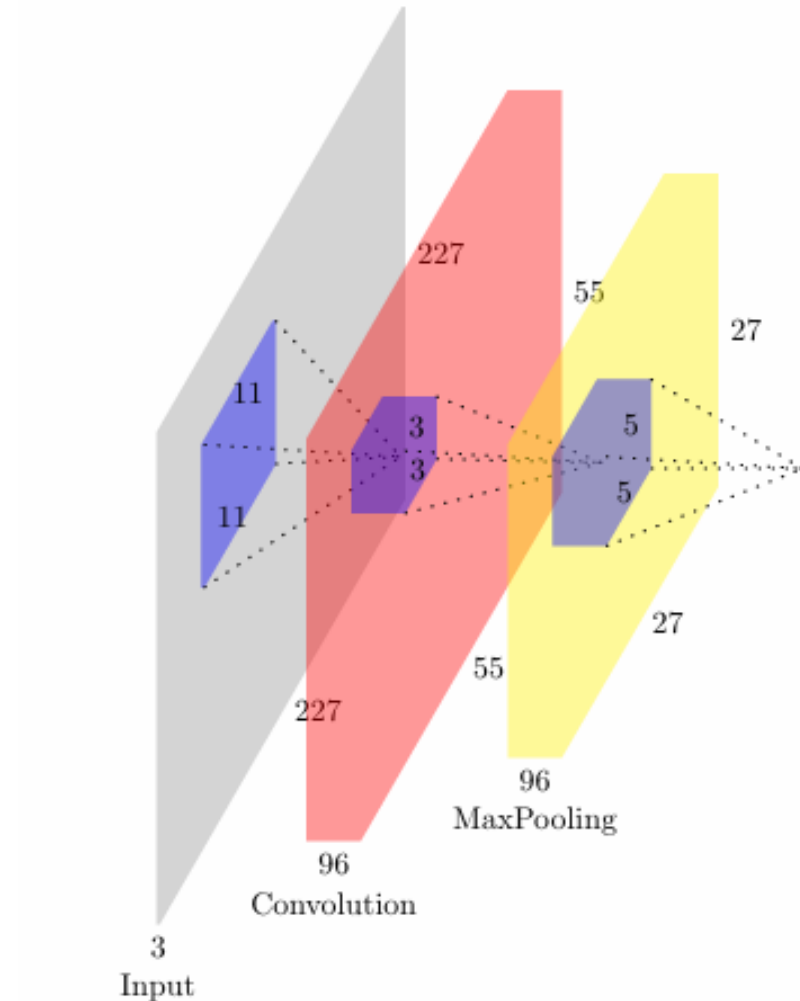
ImageNet Large Scale Visual Recognition Challenge 2012

AlexNet



Input: $227 \times 227 \times 3$
Conv1: $K = 96, F = 11$
 $S = 4, P = 0$
Output: $W_2 = 55, H_2 = 55$
Parameters: $(11 \times 11 \times 3) \times 96 = 34K$

AlexNet



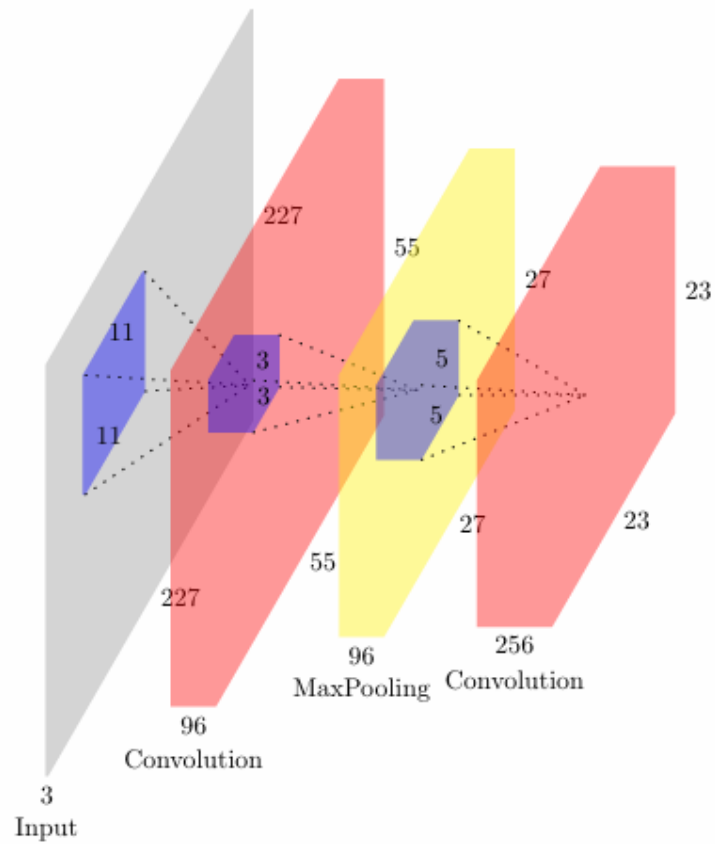
Max Pool Input: $55 \times 55 \times 96$

$F = 3, S = 2$

Output: $W_2 = 27, H_2 = 27$

Parameters: 0

AlexNet



Input: $27 \times 27 \times 96$
 Conv1: $K = 256, F = 5$
 $S = 1, P = 0$
 Output: $W_2 = 23, H_2 = 23$
 Parameters: $(5 \times 5 \times 96) \times 256 = 0.6M$

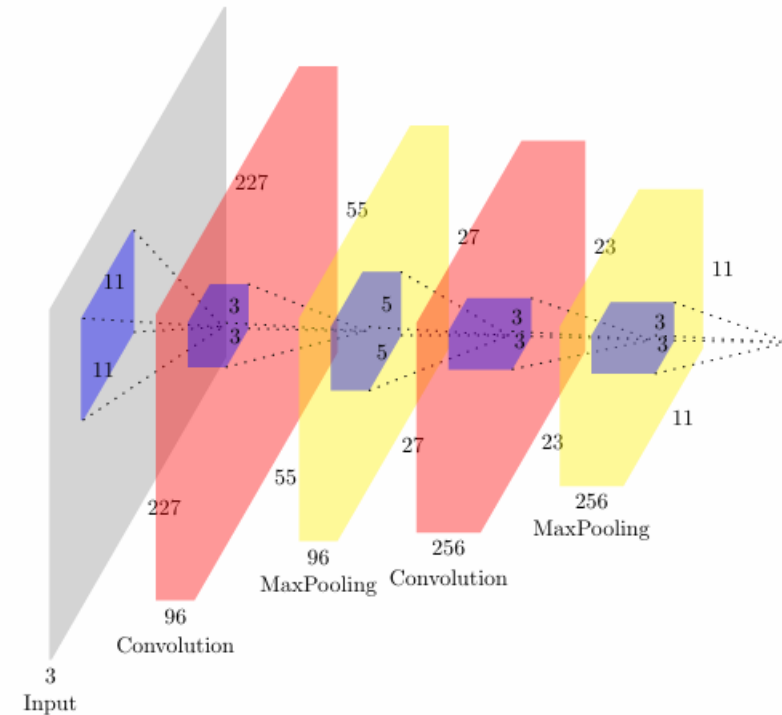
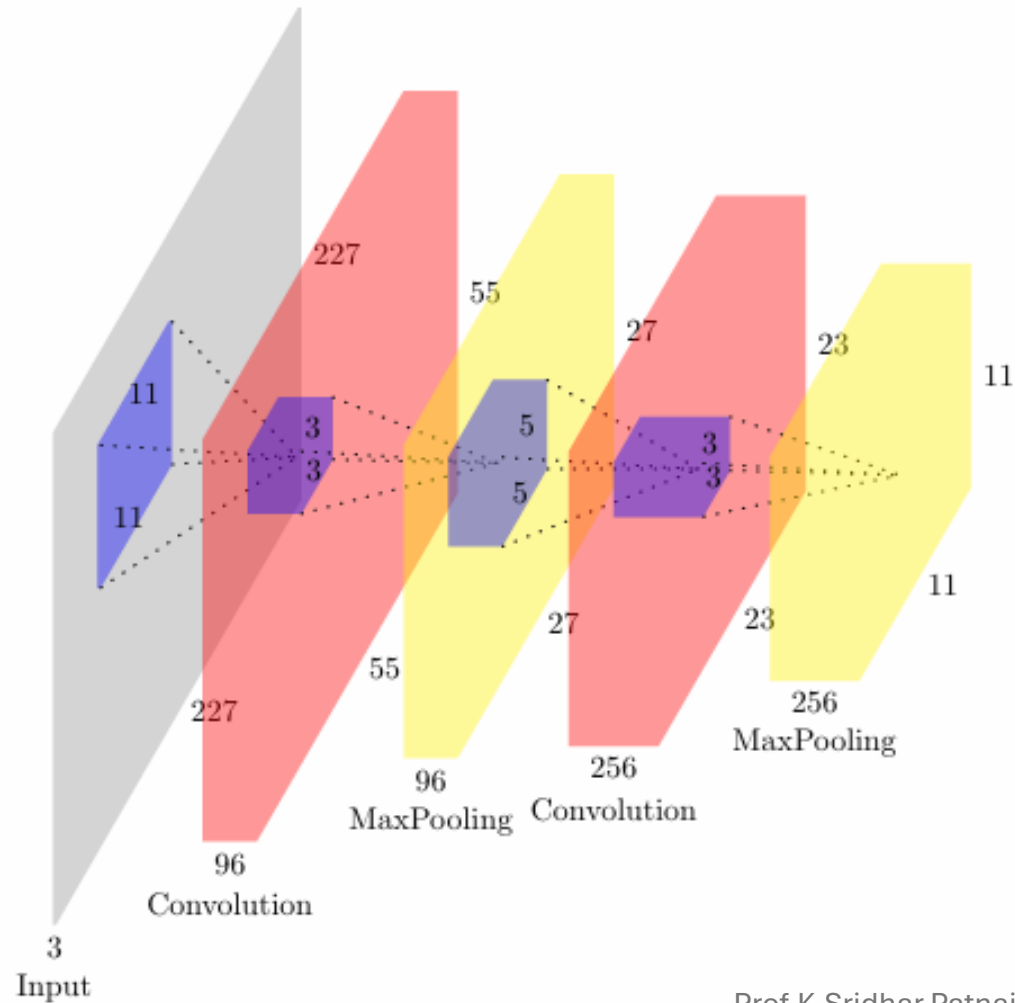
AlexNet

Max Pool Input: $23 \times 23 \times 256$

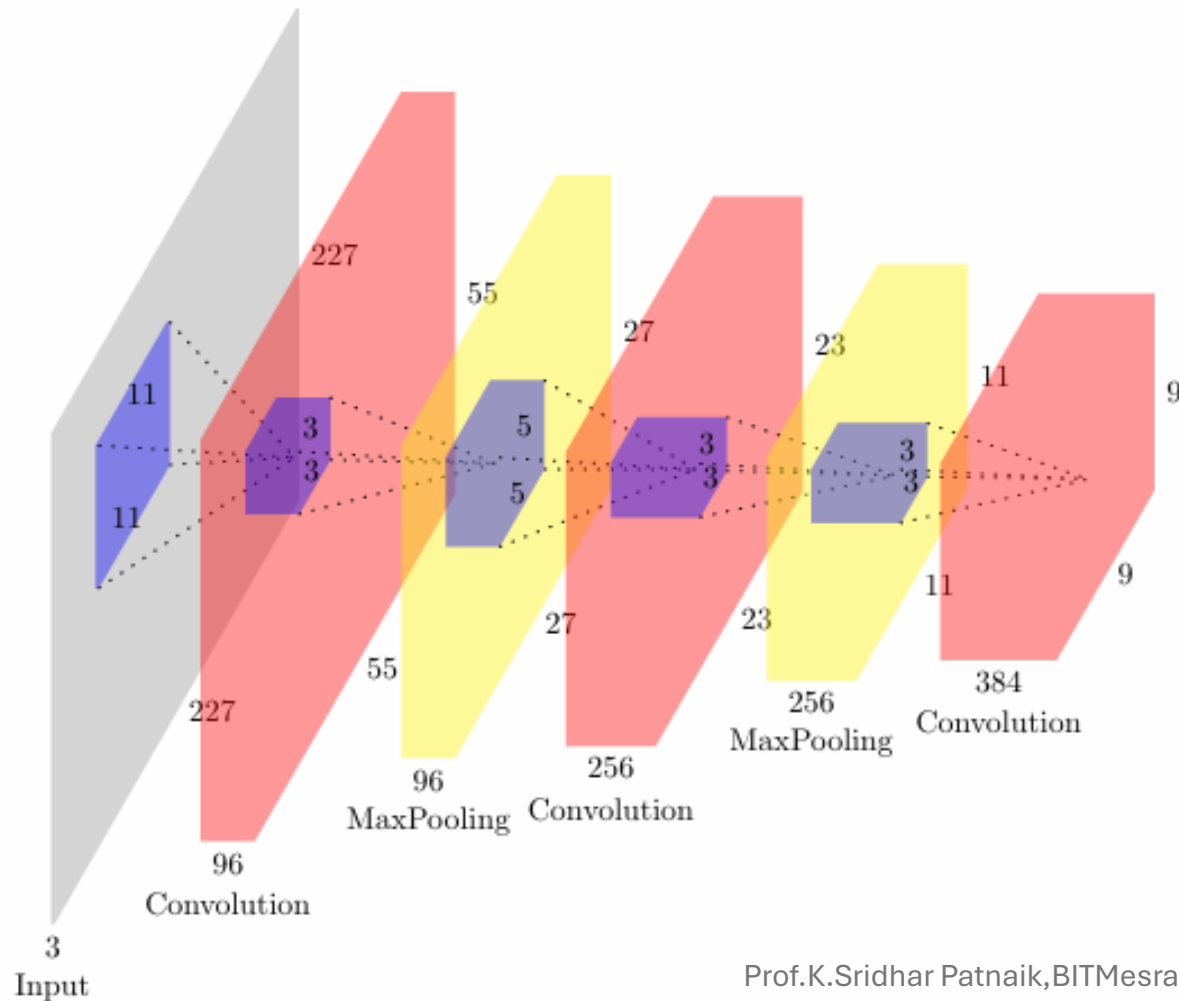
$$F = 3, S = 2$$

Output: $W_2 = 11, H_2 = 11$

Parameters: 0

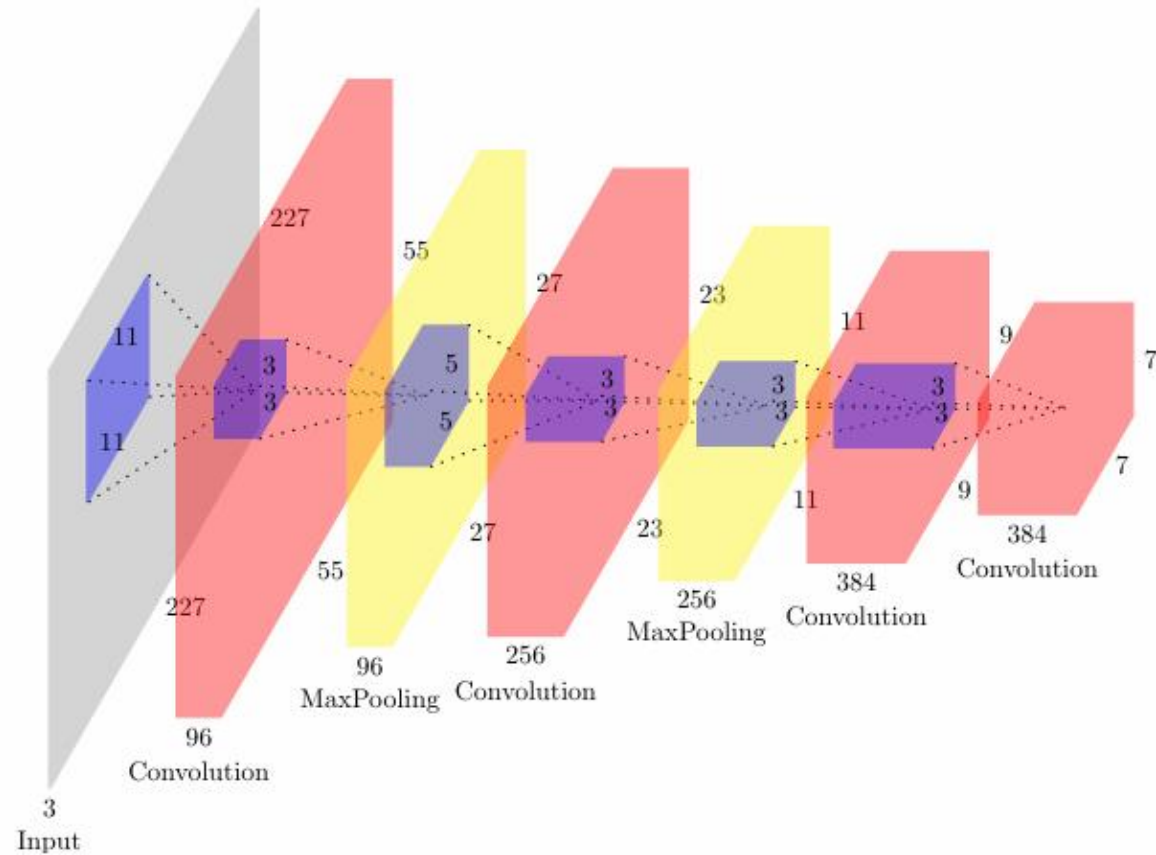


AlexNet



Input: $11 \times 11 \times 256$
 Conv1: $K = 384, F = 3$
 $S = 1, P = 0$
 Output: $W_2 = 9, H_2 = 9$
 Parameters: $(3 \times 3 \times 256) \times 384 = 0.8M$

AlexNet



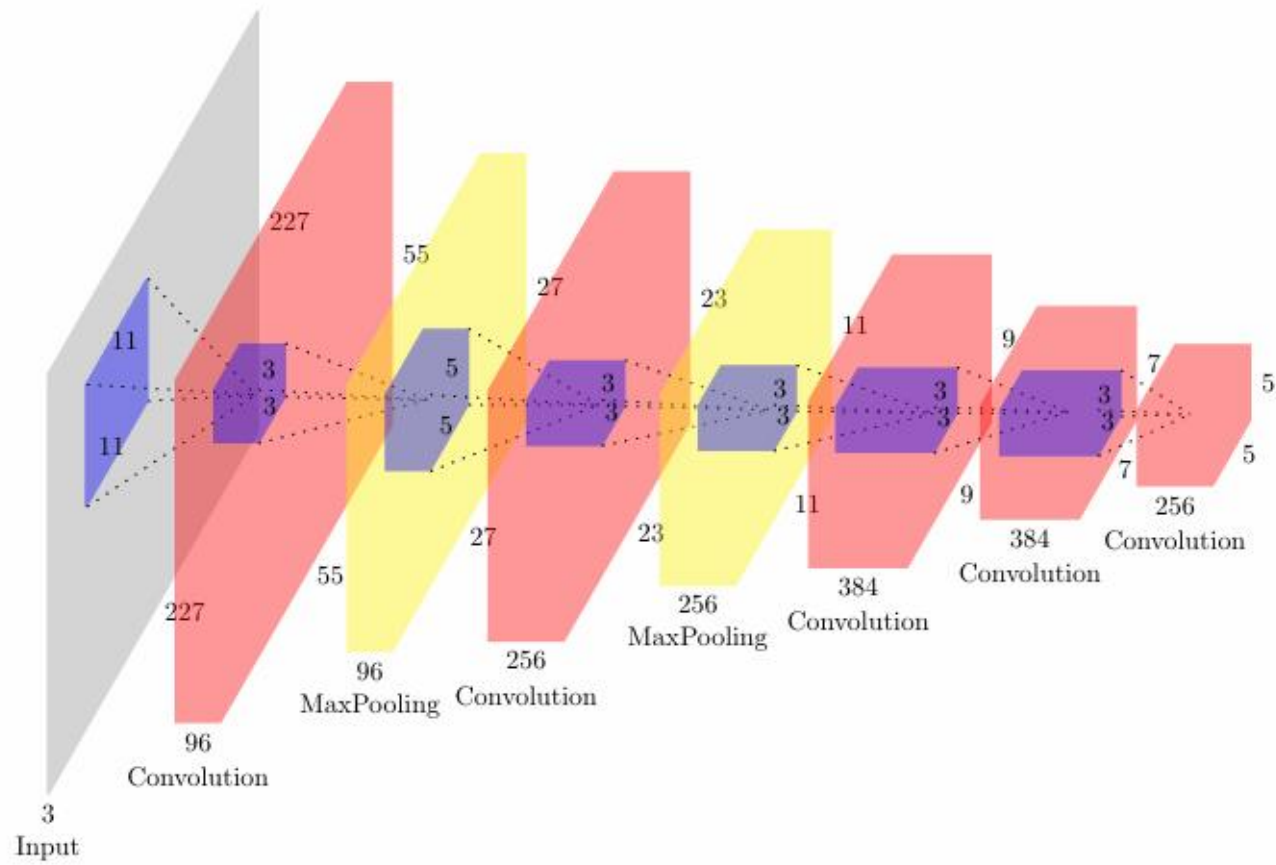
Input: $9 \times 9 \times 384$
 Conv1: $K = 384, F = 3$
 $S = 1, P = 0$
 Output: $W_2 = 7, H_2 = 7$
 Parameters: ?

Diagram illustrating a 3D CNN architecture for handwritten digit recognition. The input is a 3x11x11 volume. The architecture consists of the following layers and operations:

- Input:** 3x11x11 (Blue)
- Layer 1:** Convolution (Red) with 96 filters of size 3x3x3, resulting in a 227x55x96 volume.
- Layer 2:** MaxPooling (Yellow) with a 2x2x2 kernel, resulting in a 27x27x27 volume.
- Layer 3:** Convolution (Red) with 256 filters of size 3x3x3, resulting in a 23x23x256 volume.
- Layer 4:** MaxPooling (Yellow) with a 2x2x2 kernel, resulting in a 11x11x11 volume.
- Layer 5:** Convolution (Red) with 384 filters of size 3x3x3, resulting in a 9x9x384 volume.
- Layer 6:** Convolution (Red) with 384 filters of size 3x3x3, resulting in a 7x7x384 volume.

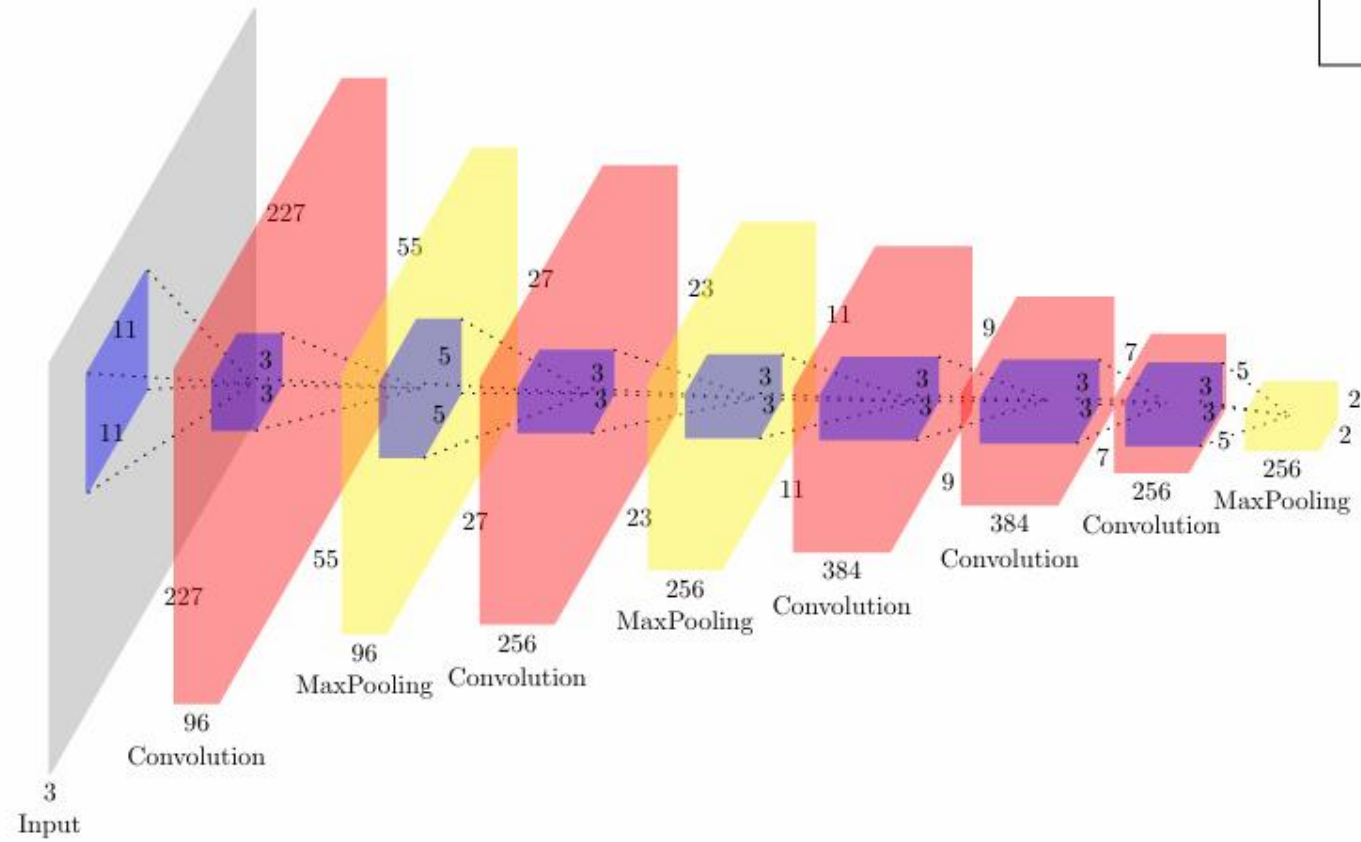
Parameters: $(3 \times 3 \times 384) \times 384 = 1.327M$

Abstract



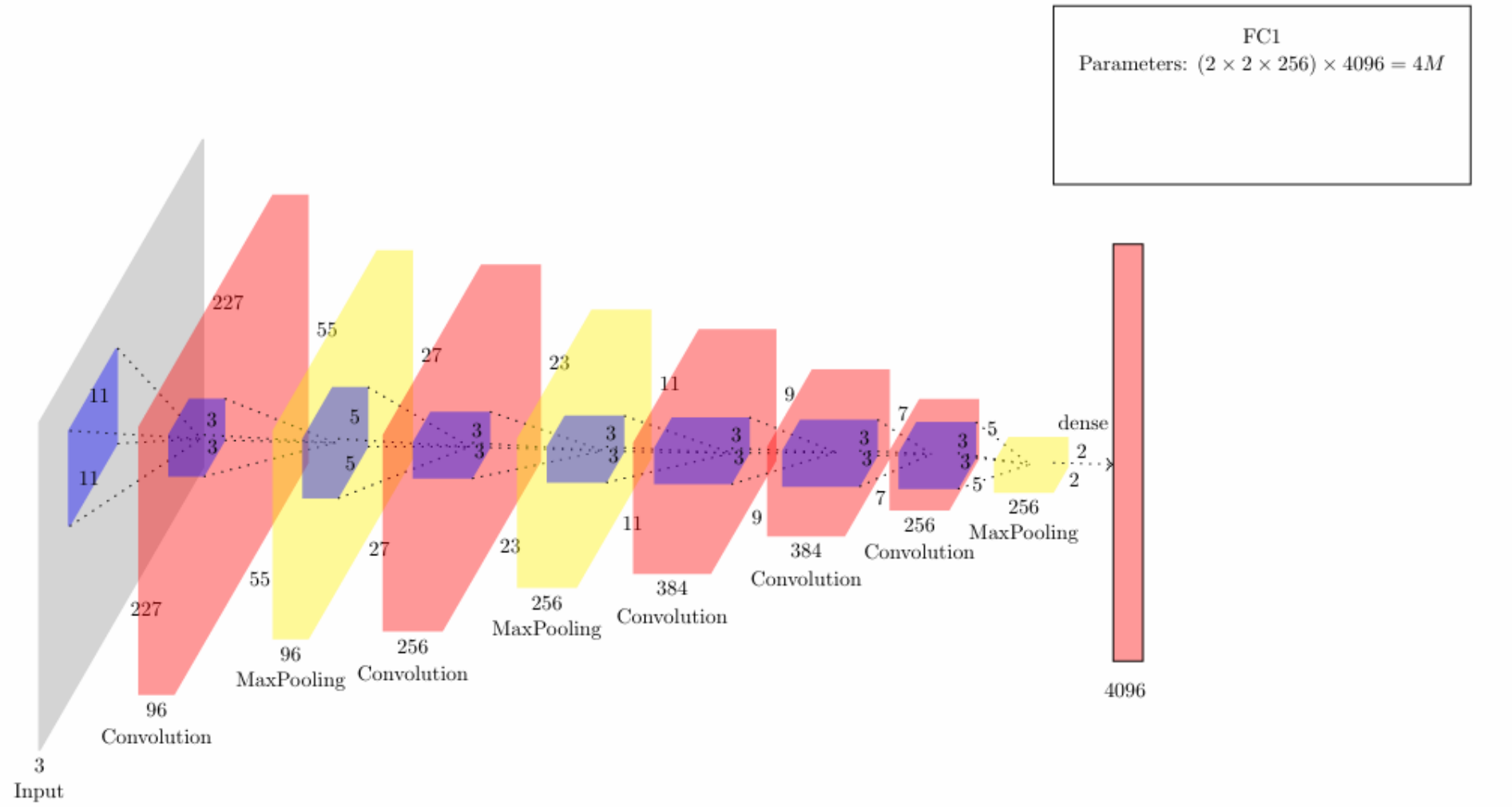
Input: $7 \times 7 \times 384$
 Conv1: $K = 256, F = 3$
 $S = 1, P = 0$
 Output: $W_2 = 5, H_2 = 5$
 Parameters: ?

AlexNet

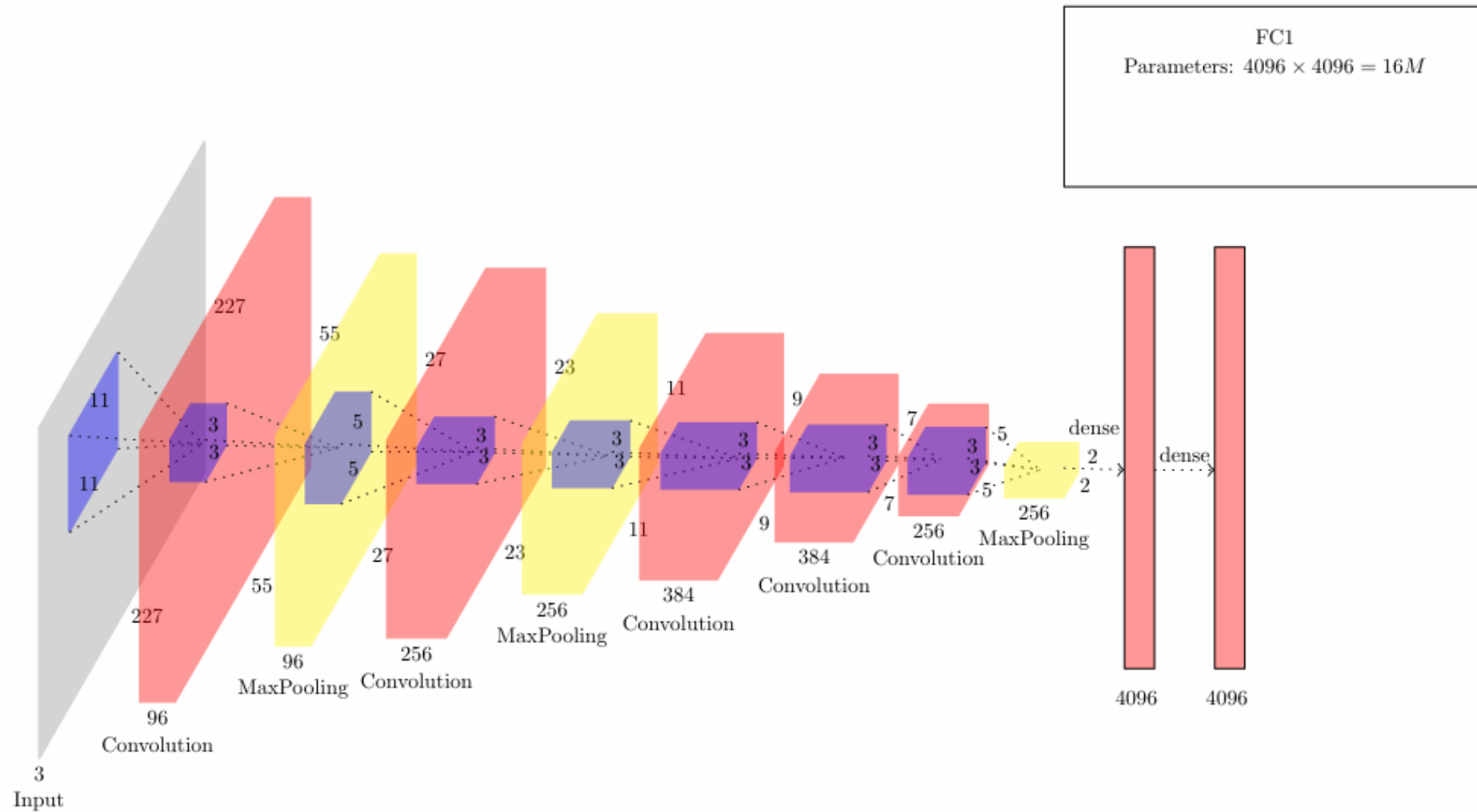


Max Pool Input: $5 \times 5 \times 256$
 $F = 3, S = 2$
 Output: $W_2 = 2, H_2 = 2$
 Parameters: ?

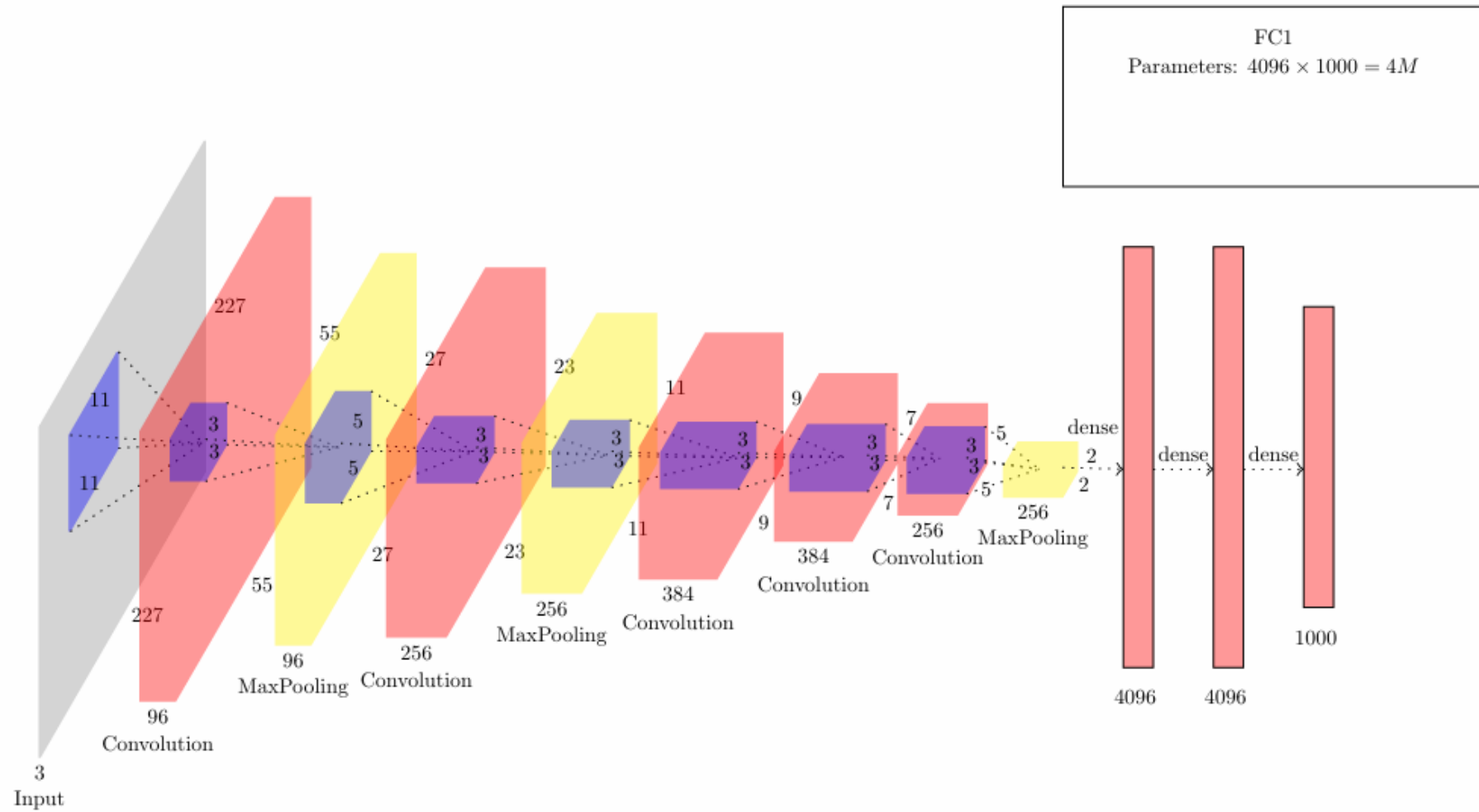
AlexNet



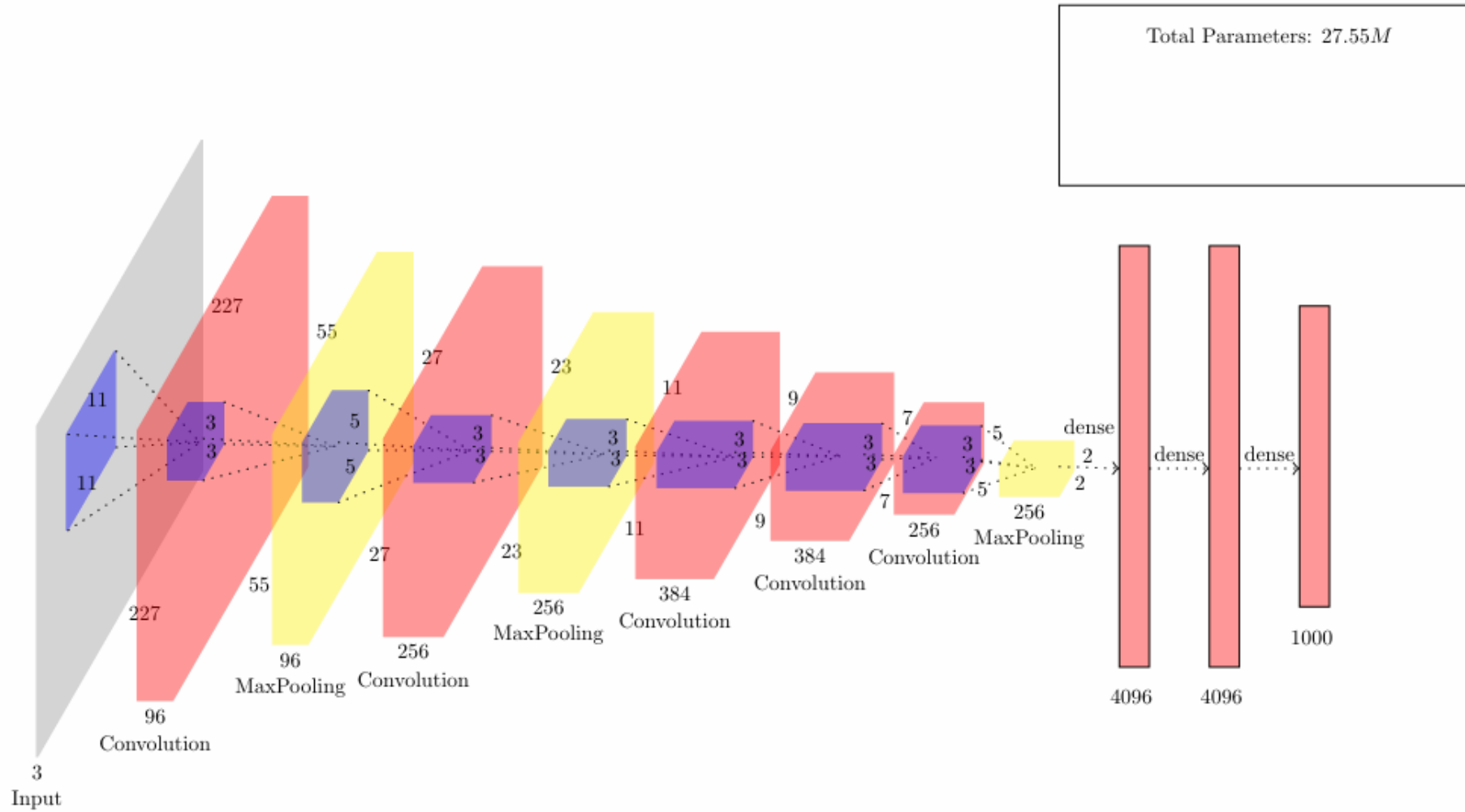
AlexNet



AlexNet

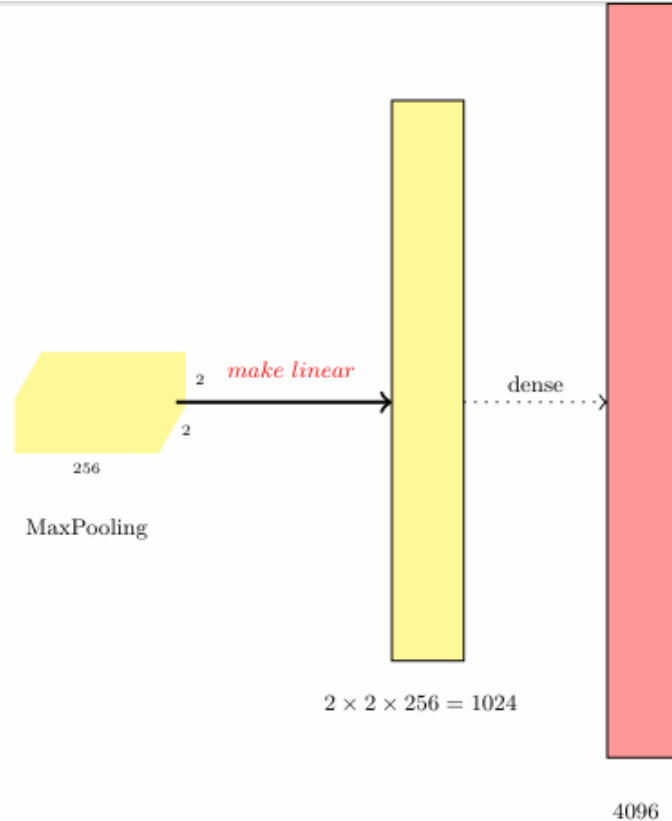


AlexNet



AlexNet

- Let us look at the connections in the fully connected layers in more detail
- We will first stretch out the last conv or maxpool layer to make it a 1d vector
- This 1d vector is then densely connected to other layers just as in a regular feedforward neural network



AlexNet

- Let us look at the connections in the fully connected layers in more detail
- We will first stretch out the last conv or maxpool layer to make it a 1d vector
- This 1d vector is then densely connected to other layers just as in a regular feedforward neural network

